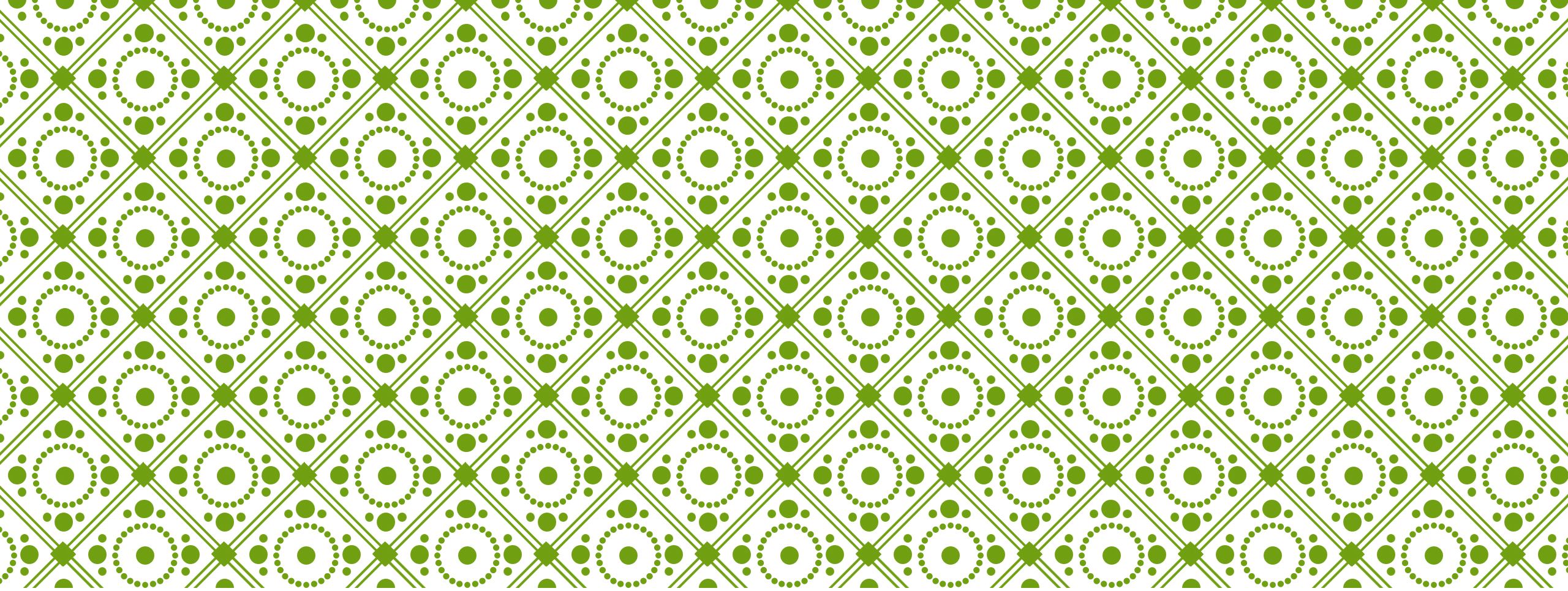




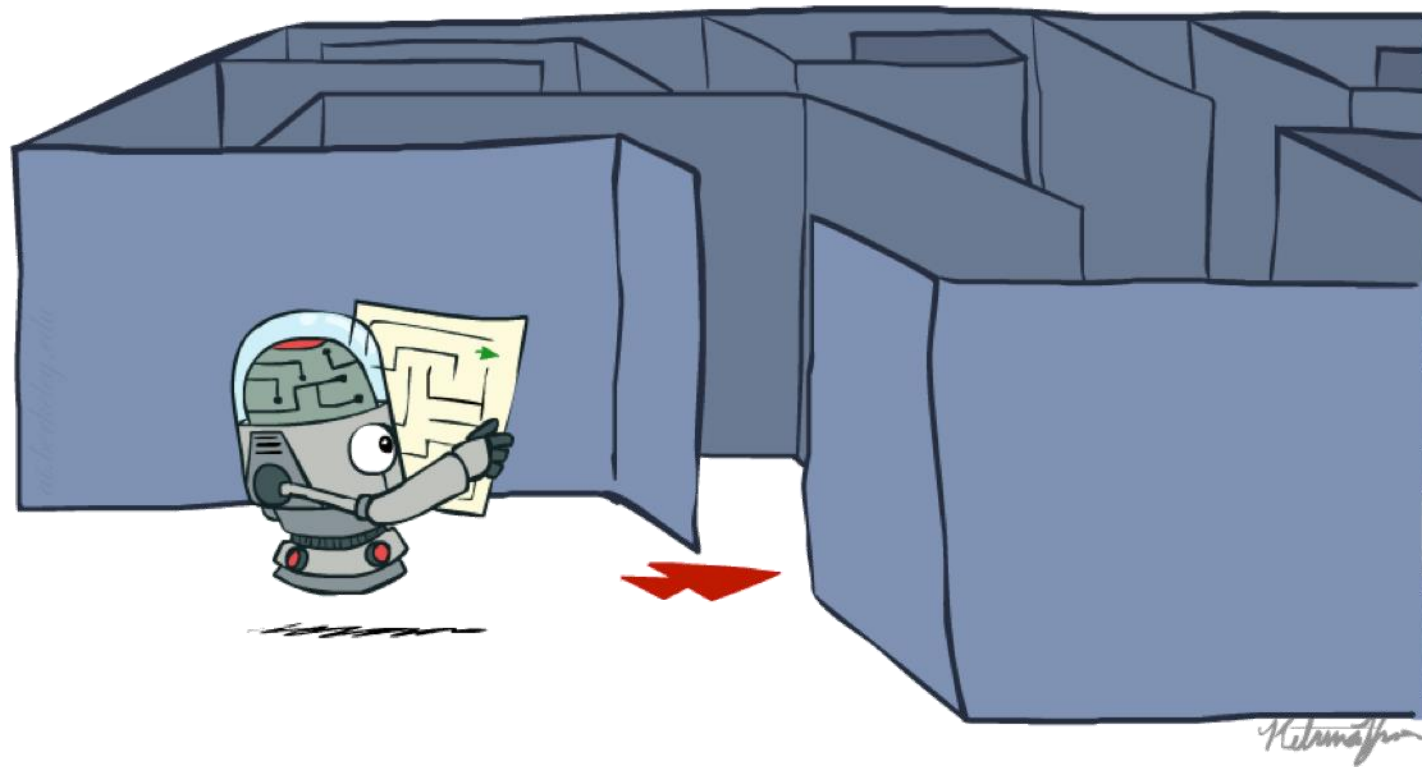
AI2002 — ARTIFICIAL INTELLIGENCE

SPRING 2024 - LECTURE 10-12

Presented By: Mr. Sandesh
Kumar

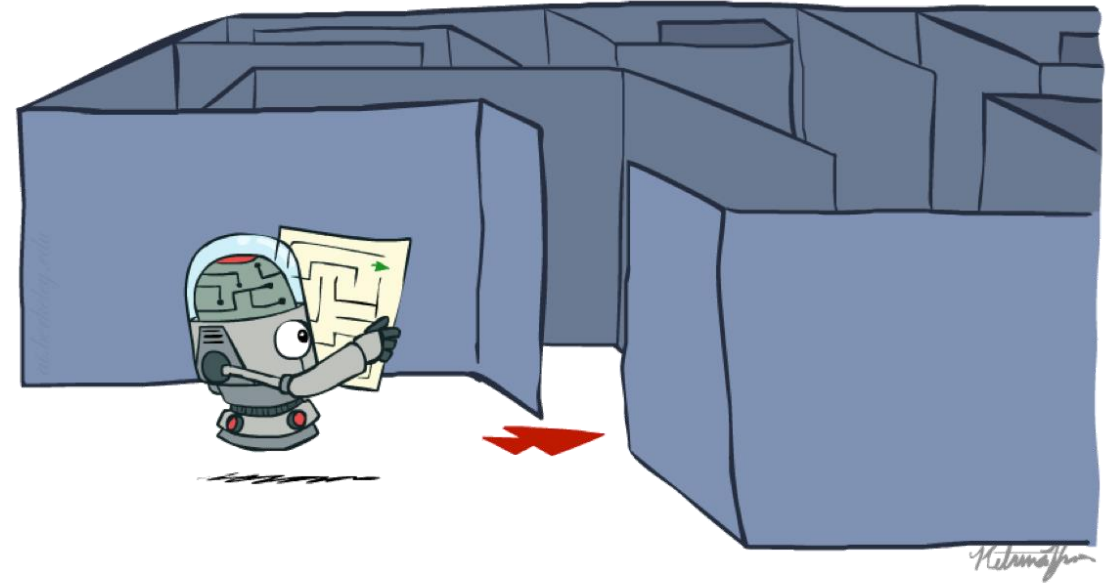
Few slides are taken from UC
Berkeley

Recap: Search

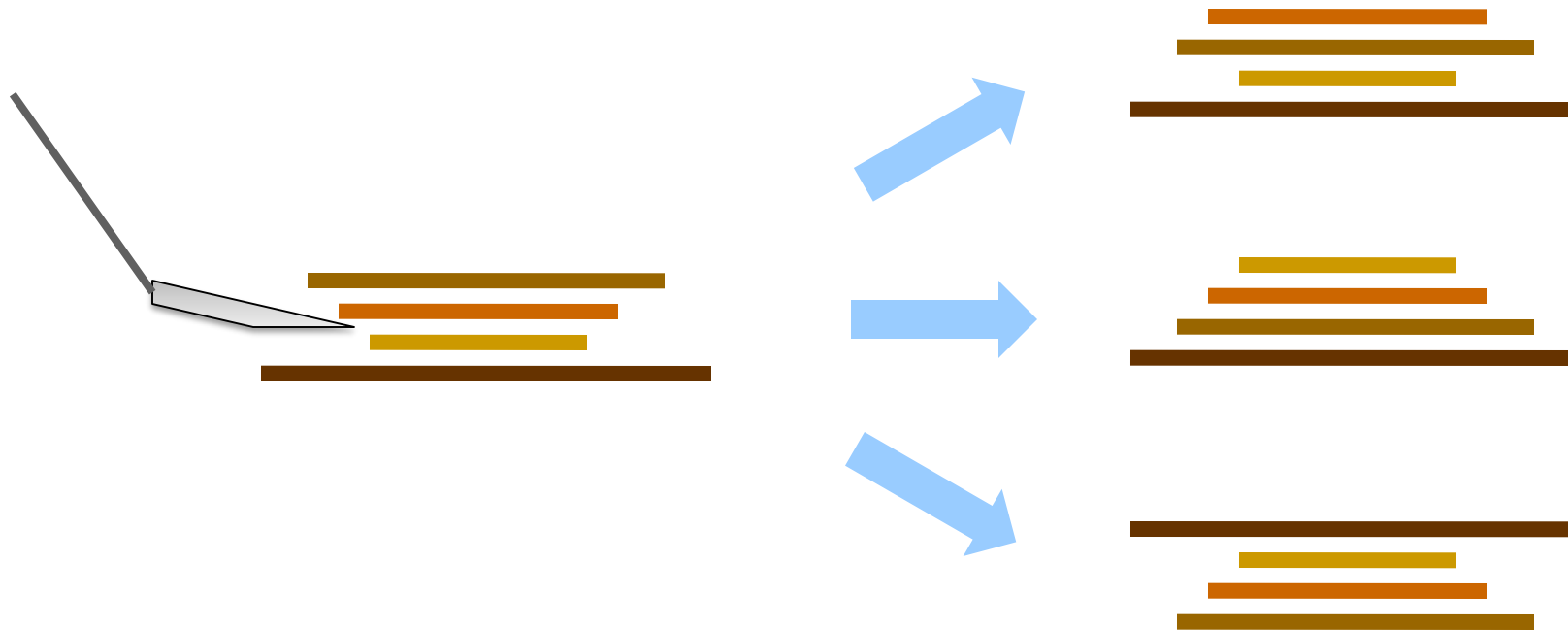


Recap: Search

- **Search problem:**
 - States (configurations of the world)
 - Actions and costs
 - Successor function (world dynamics)
 - Start state and goal test
- **Search tree:**
 - Nodes: represent plans for reaching states
 - Plans have costs (sum of action costs)
- **Search algorithm:**
 - Systematically builds a search tree
 - Chooses an ordering of the fringe (unexplored nodes)
 - Optimal: finds least-cost plans



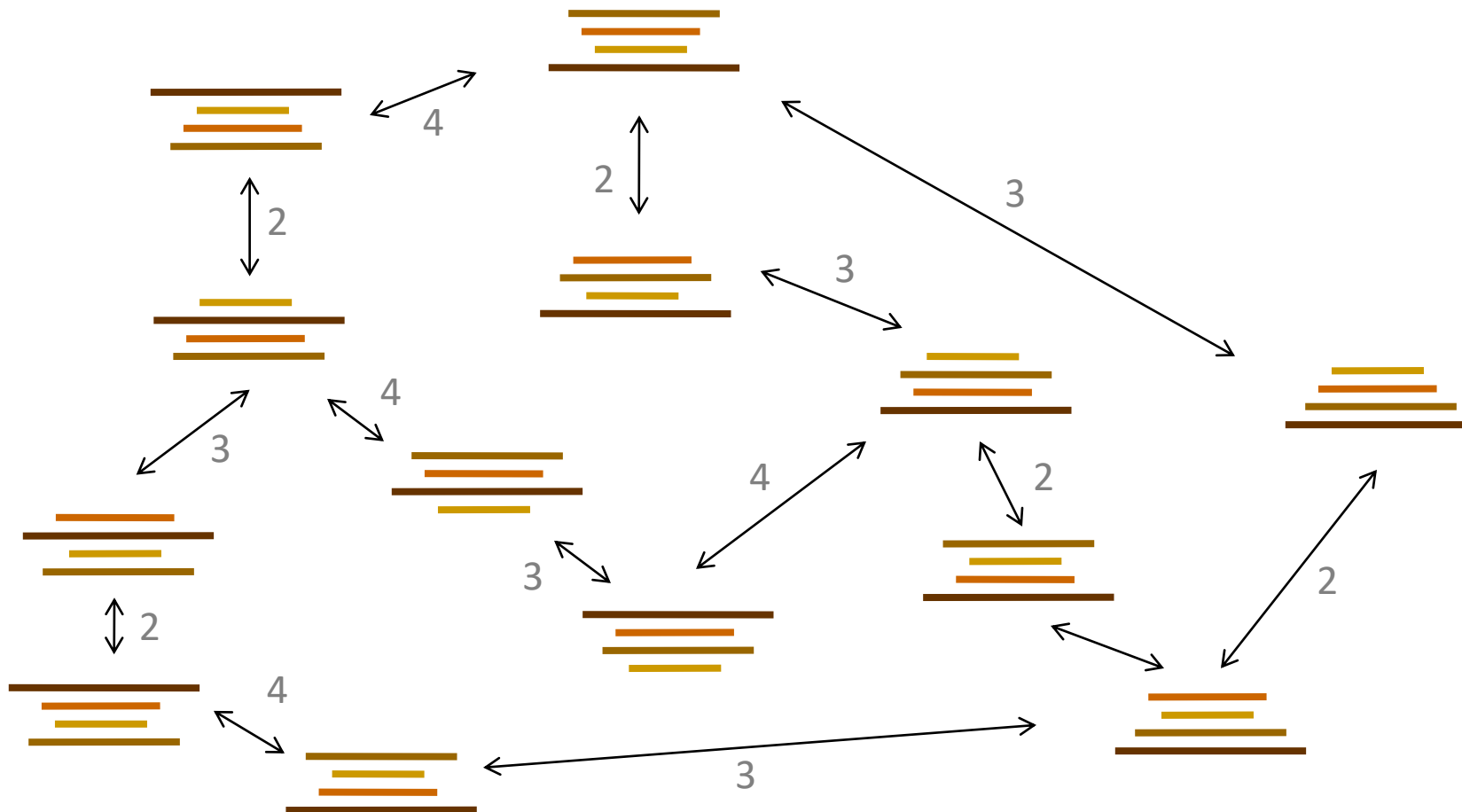
Example: Pancake Problem



Cost: Number of pancakes flipped

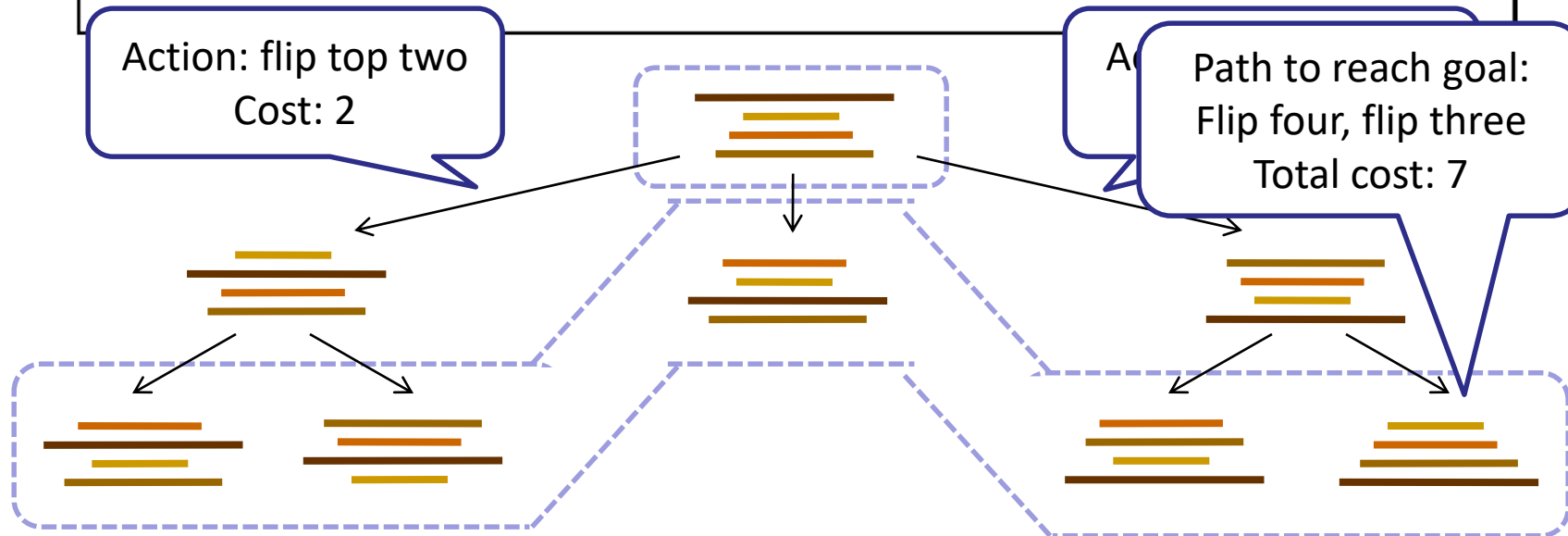
Example: Pancake Problem

State space graph with costs as weights

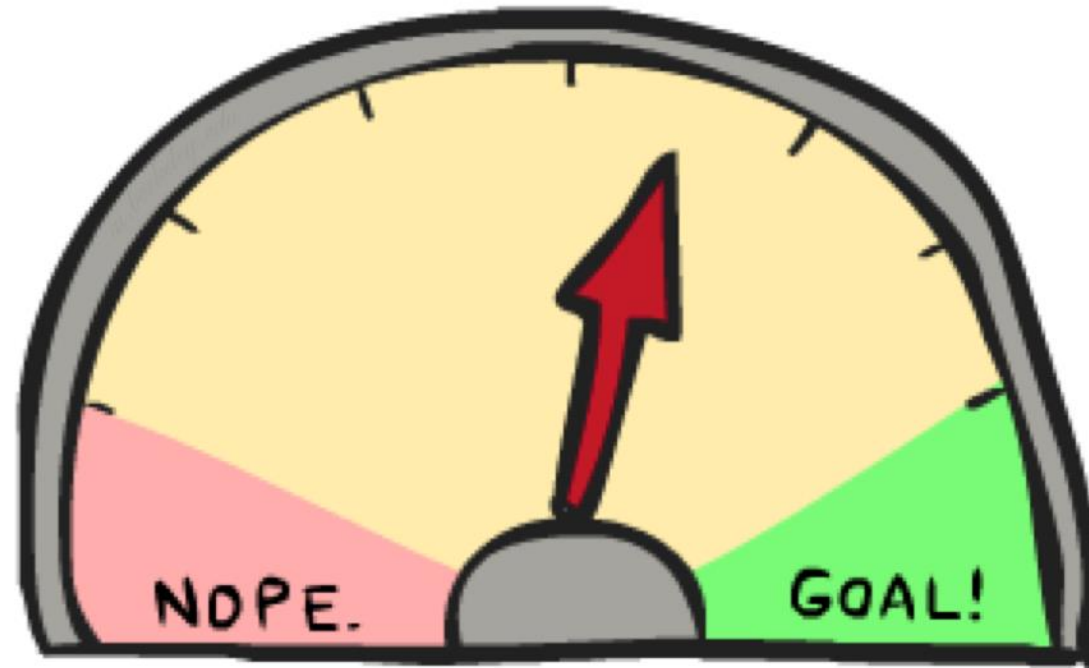


General Tree Search

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```



Informed Search



Informed (Heuristic) Search Strategies

- An **informed search strategy** uses problem specific knowledge beyond the definition of the problem itself and it can find solutions more efficiently than can an *uninformed strategy*.
- **Best-first search** is an algorithm in which a node is selected for expansion based on an **evaluation function, $f(n)$** .
- The evaluation function is construed as a cost estimate, so the node with the lowest evaluation is expanded first.
- The implementation of best-first graph search is identical to that for **uniform-cost search** except for the use of *evaluation function f* instead of *lowest path cost g* to order the priority queue.
- The choice of *evaluation function* determines the search strategy.
- Most **best-first algorithms** include a **heuristic function, $h(n)$** as a component of *evaluation function*.

Uniform-Cost Search: Review

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

frontier $\leftarrow$ INSERT(*child*, *frontier*)

else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

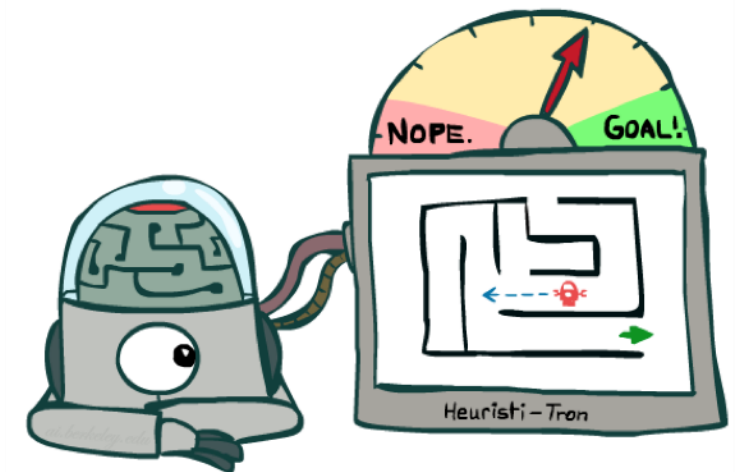
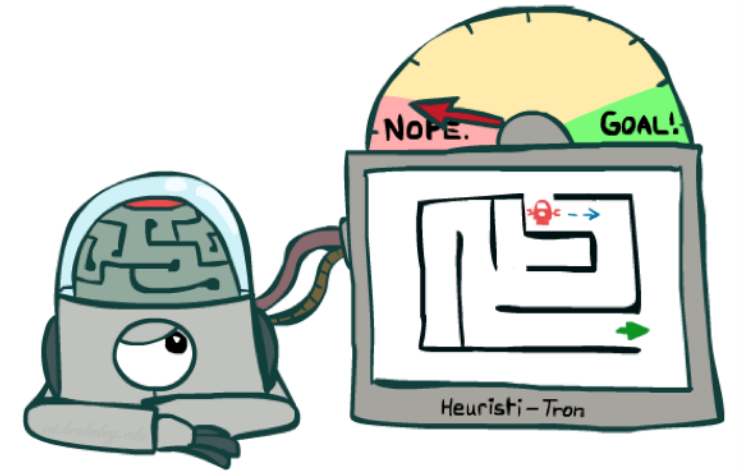
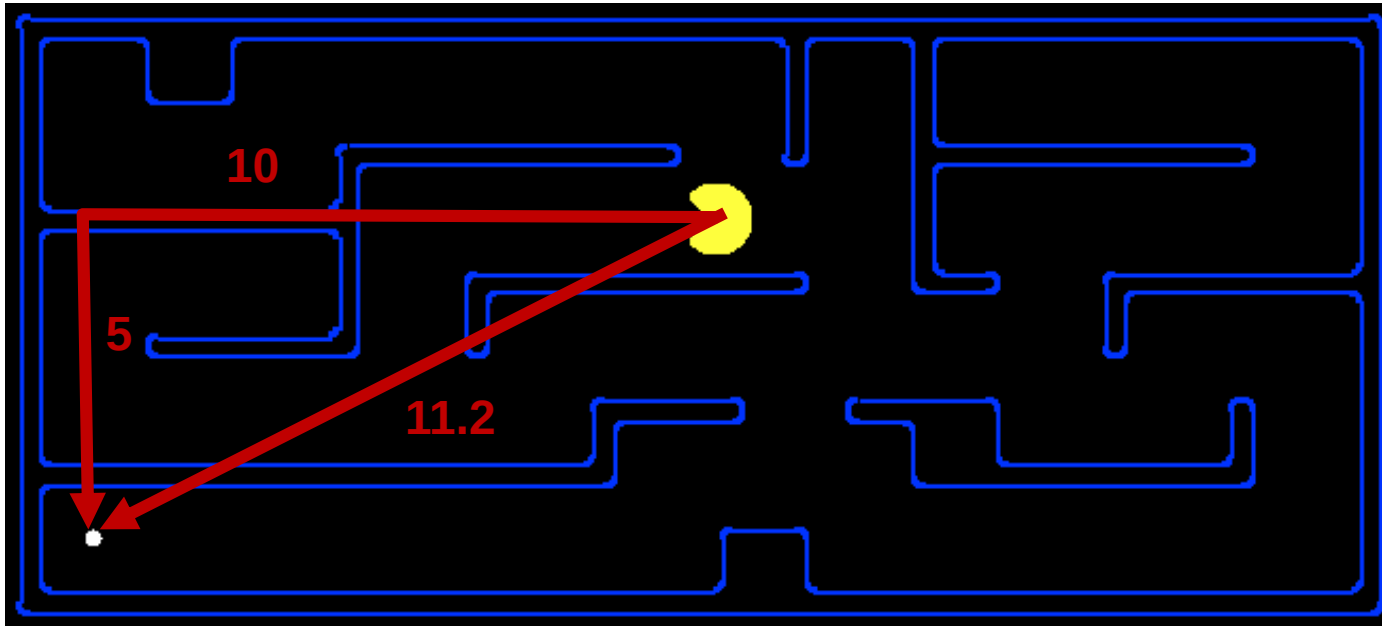
We have use evaluation function $f(n)$ instead of PATH-COST for Best-first search.

Heuristic Function

- A **heuristic function $h(n)$** is the estimated cost of the *cheapest path* from the state at node n to a goal state.
- **Heuristic functions** are the most common form of additional knowledge of the problem for *informed (heuristic) search algorithms*.
- **Heuristic functions** are nonnegative, problem-specific functions, with one constraint: if n is a goal node, then $h(n)=0$.
- When we use a **heuristic function** to guide our search, we perform **informed (heuristic”) search**.
- Some informed (heuristic) searches:
 - Greedy best-first search
 - A^*
 - Recursive best-first search (a memory-bounded heuristic search)

Search Heuristics

- A heuristic is:
 - A function that *estimates* how close a state is to a goal
 - Designed for a particular search problem
 - Examples: Manhattan distance, Euclidean distance for pathing



Heuristic Function Example

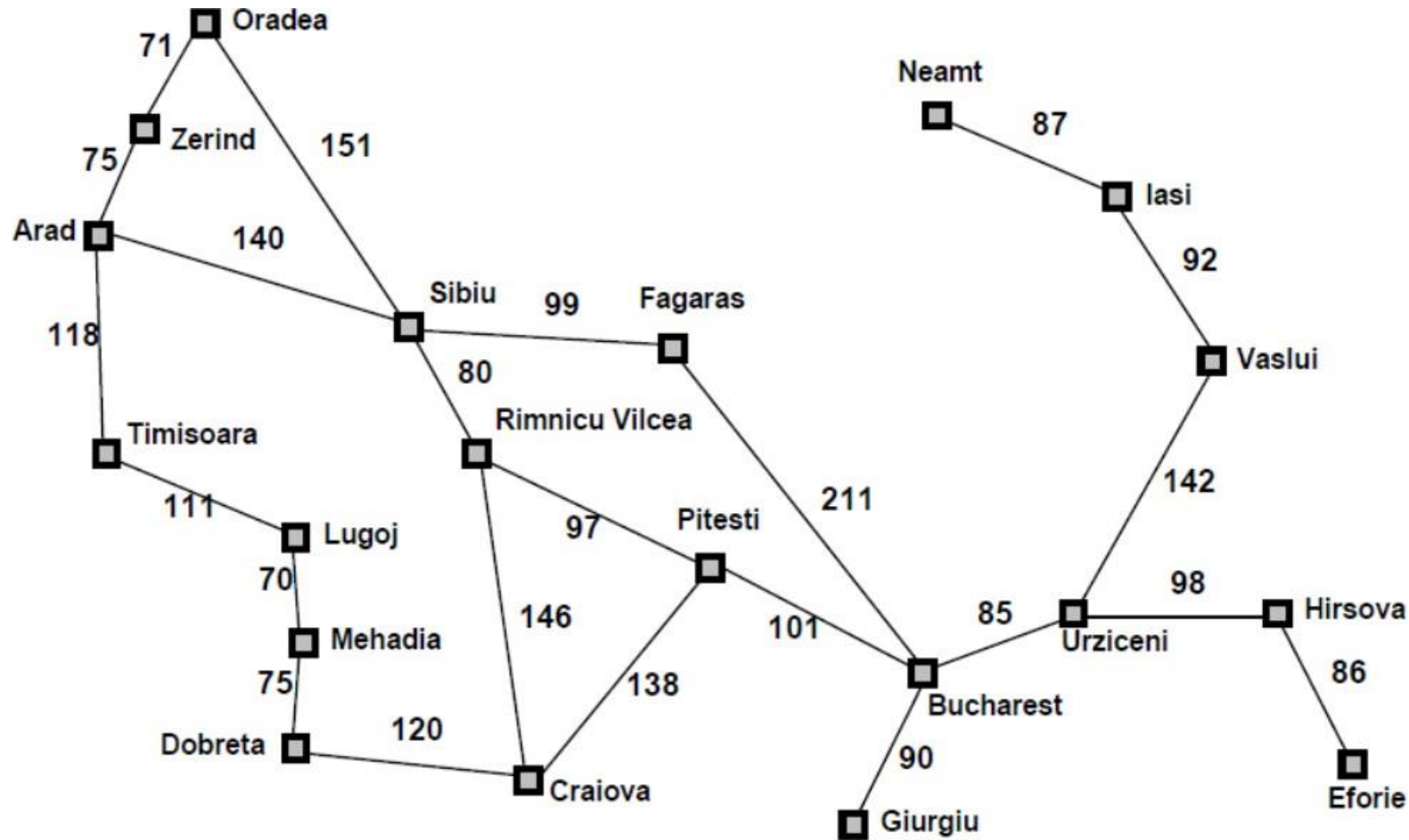
straight line distance heuristic

- For route-finding problems in Romania, we can use the **straight line distance heuristic** h_{SLD} .
- If the goal is Bucharest, we need to know the *straight-line distances to Bucharest*
- The values of h_{SLD} cannot be computed from the problem description itself.
- Since h_{SLD} is correlated with actual road distances and it is a useful heuristic.

$h_{\text{SLD}}(n)$ = **straight-line distance from node n to Bucharest**

Heuristic Function Example

straight line distance heuristic

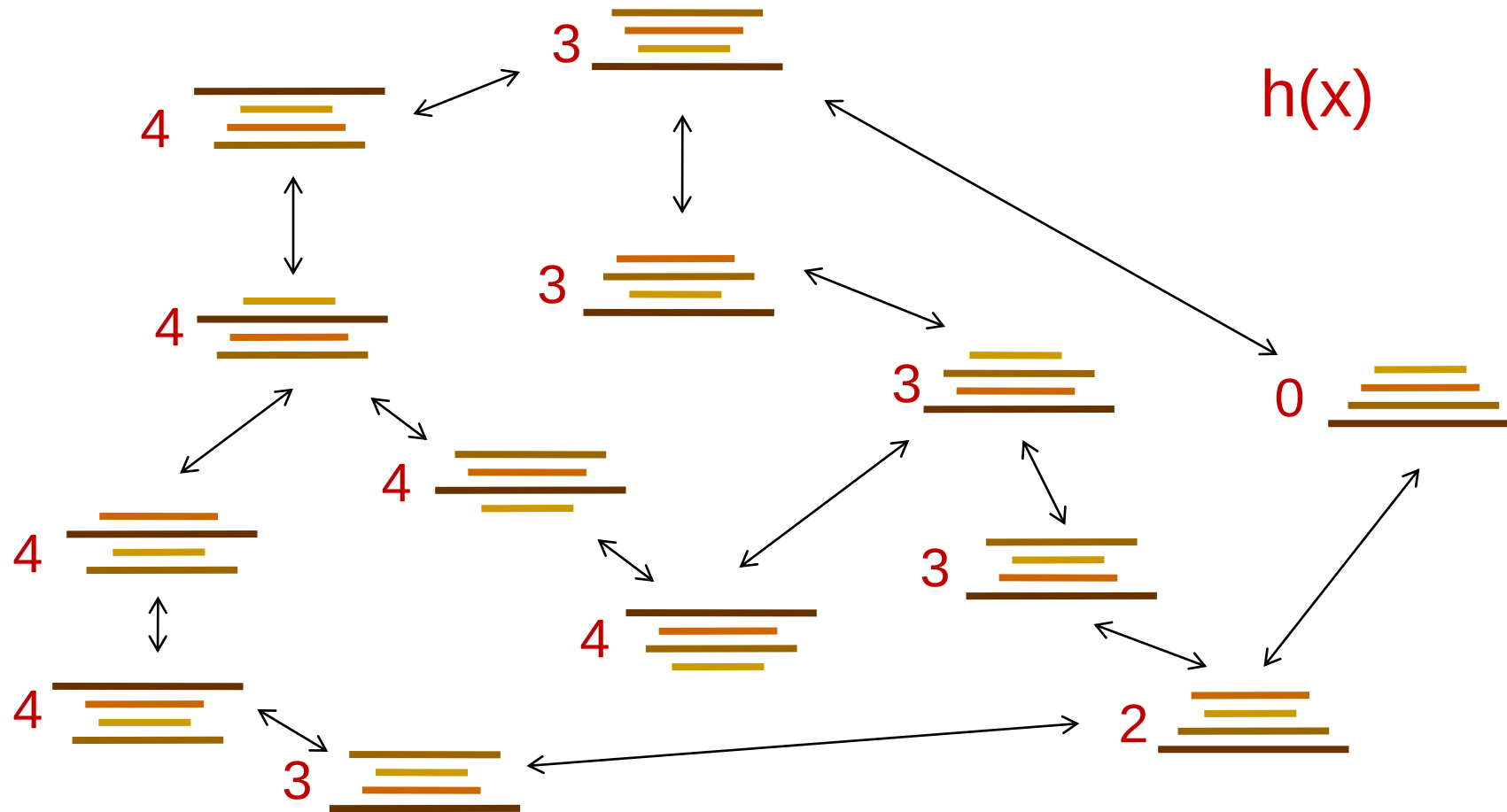


*straight-line distances
to Bucharest*

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example: Heuristic Function

Heuristic: the number of the largest pancake that is still out of place





Greedy best-first search

- **Greedy best-first search** expands the node that is *closest to the goal*, on the grounds that this is likely to lead to a solution quickly.
- Greedy search expands the node that appears to be closest to goal
- **Greedy best-first search** evaluates nodes by using just *the heuristic function*.
- This means that it uses *heuristic function* $h(n)$ as the *evaluation function* $f(n)$ (that is $\mathbf{f(n) = h(n)}$).
- For Romania problem, the heuristic function
$$h_{\text{SLD}}(n) = \text{straight-line distance from } n \text{ to Bucharest}$$
as evaluation function

Greedy search example

Node labels are h_{SLD} values

Arad is the initial state.



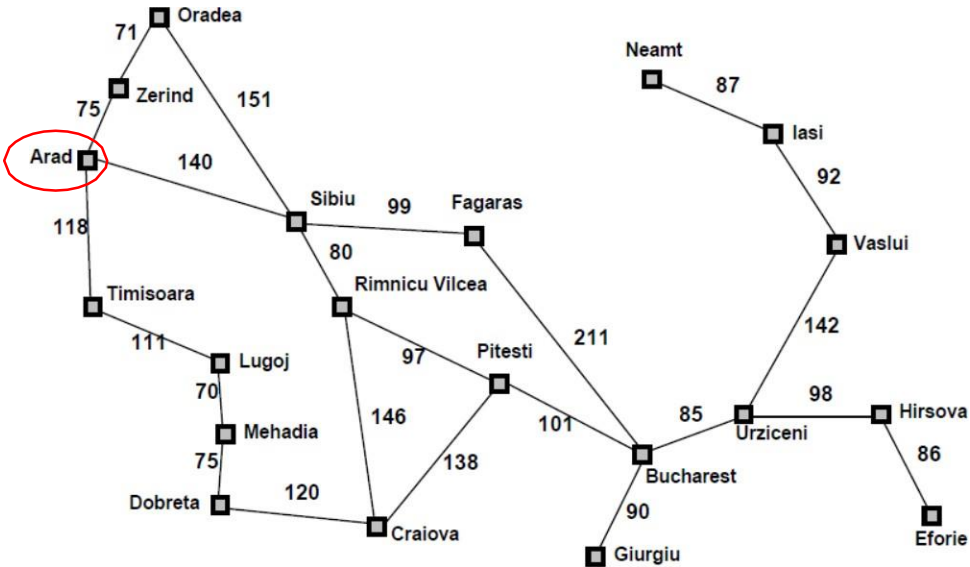
straight-line distances
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

Arad 366

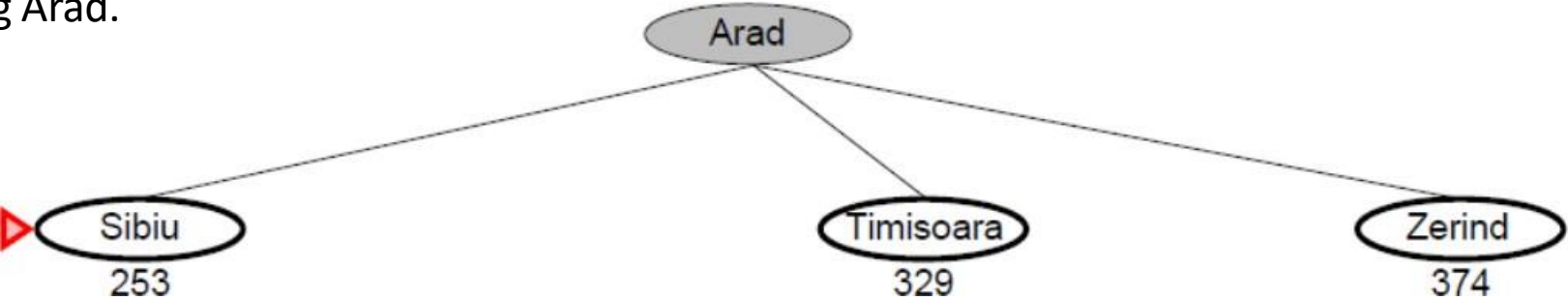
Explored



Greedy search example

Node labels are h_{SLD} values

After expanding Arad.



straight-line distances
to Bucharest

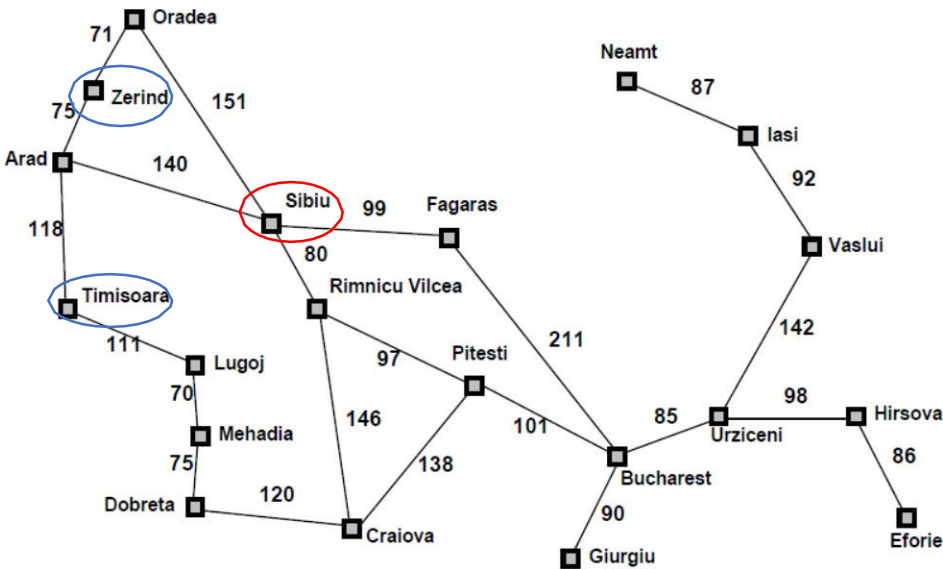
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

Sibiu	253
Timisoara	329
Zerind	374

Explored

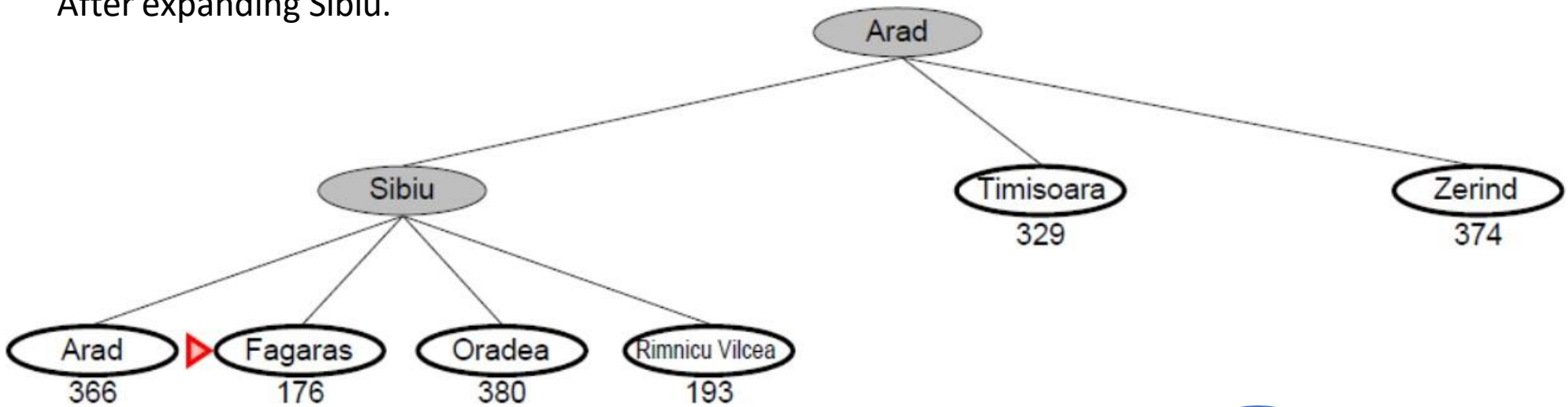
Arad



Greedy search example

Node labels are h_{SLD} values

After expanding Sibiu.



Frontier

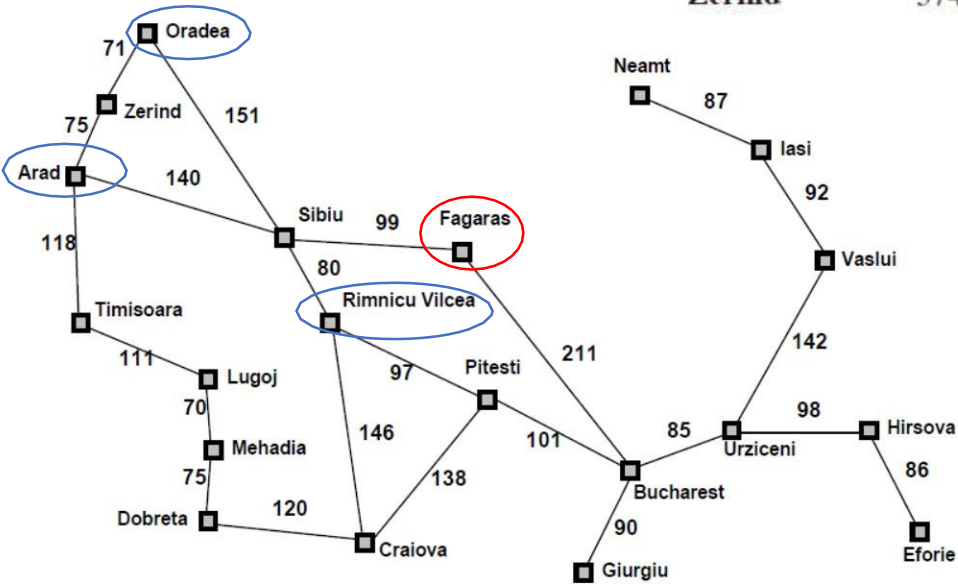
Fagaras 176
RimnicuVil 193
Timisoara 329
Zerind 374
Oradea 380

Explored

Arad
Sibiu

straight-line distances
to Bucharest

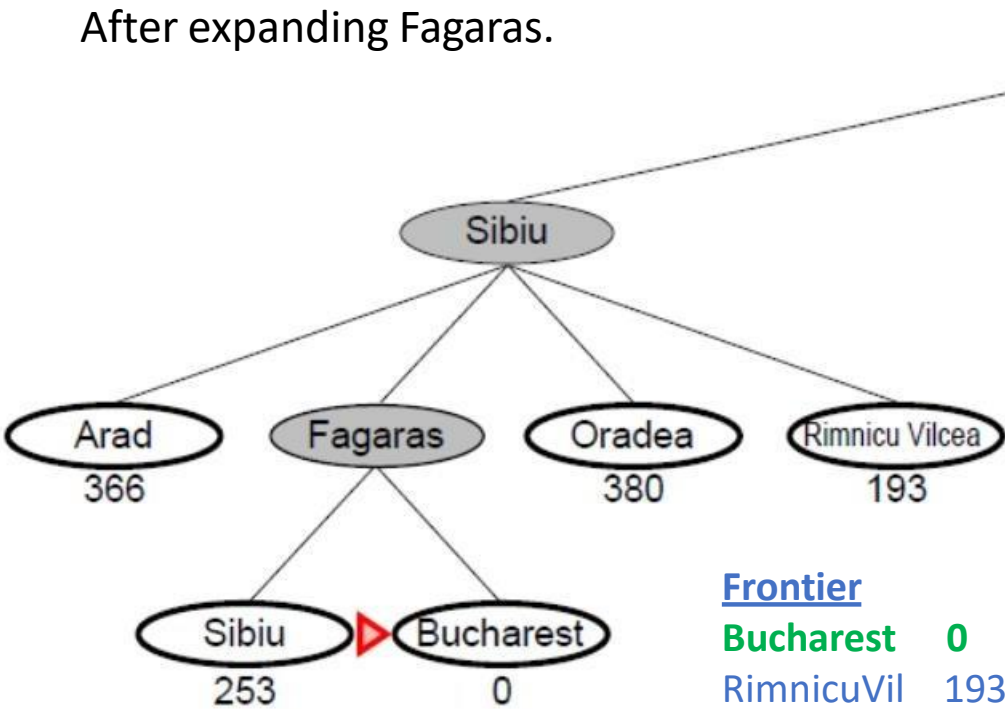
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Greedy search example

Node labels are h_{SLD} values

After expanding Fagaras.

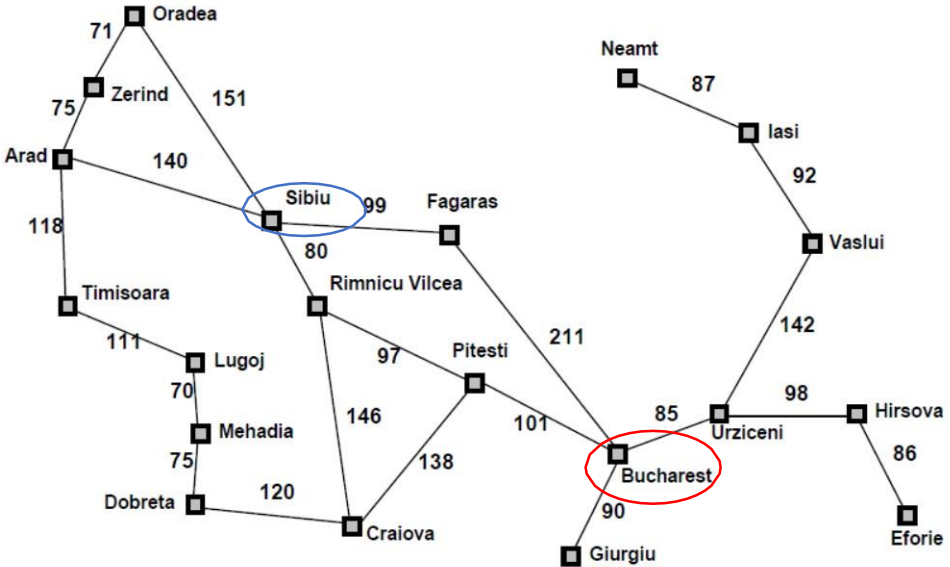


Frontier	
Bucharest	0
RimnicuVil	193
Timisoara	329
Zerind	374
Oradea	380

Explored	
Arad	
Sibiu	
Fagaras	

straight-line distances
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



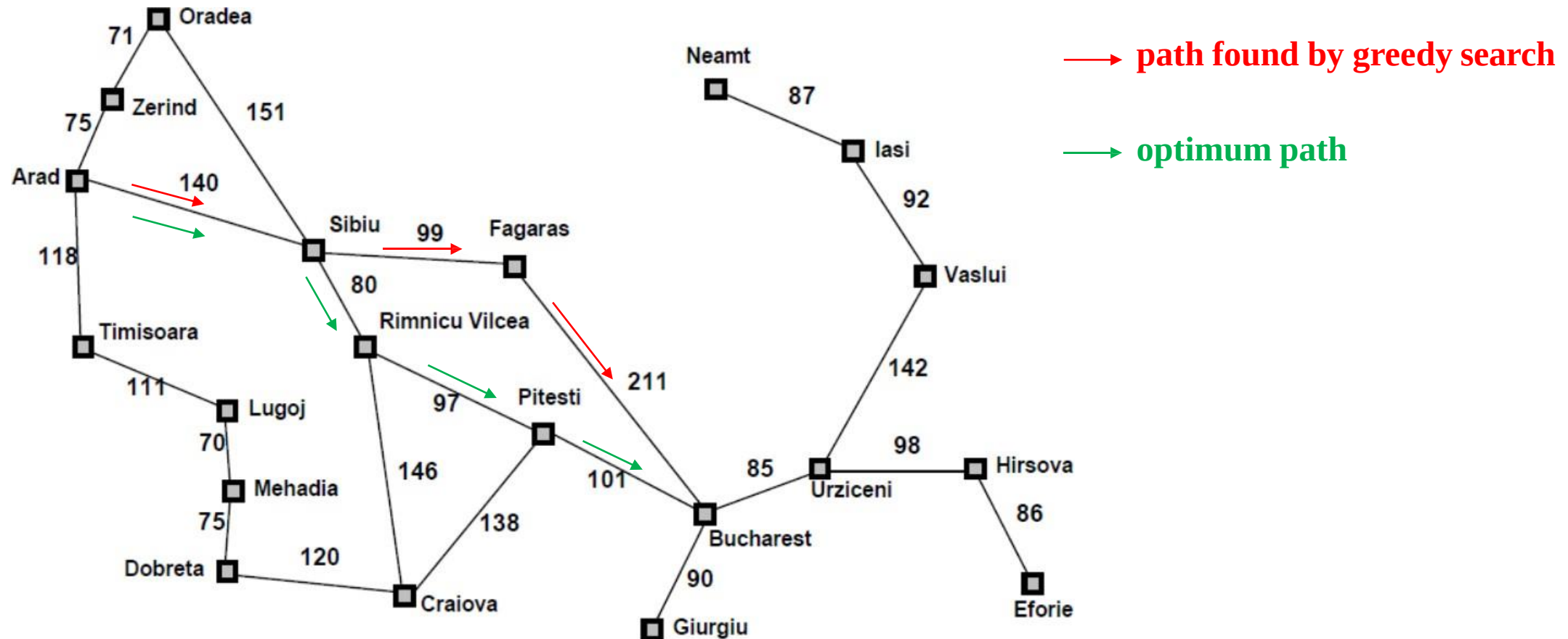
Properties of greedy search

Complete?	NO It can get stuck in loops without repeated-state checking Complete in finite space with repeated-state checking
Time?	$O(b^m)$ where m is the maximum depth of the search tree. but a good heuristic can give dramatic improvement
Space?	$O(b^m)$ keeps all nodes in memory
Optimal?	NO nodes expanded in increasing order of path cost

Properties of greedy search

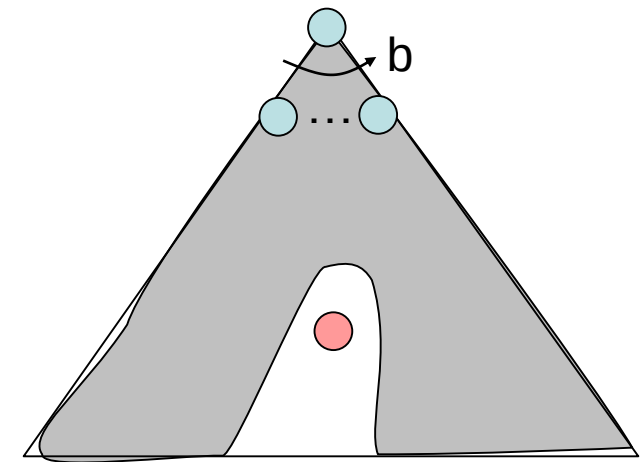
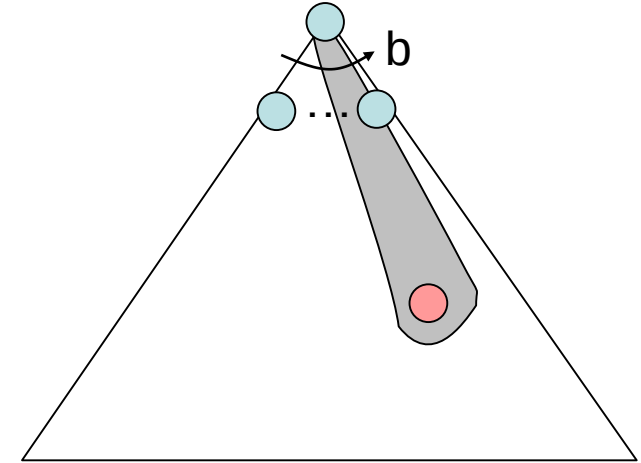
Optimal? NO

the path via Sibiu and Fagaras to Bucharest is 32 kilometers longer than the path through Rimnicu Vilcea and Pitesti.



Greedy Search

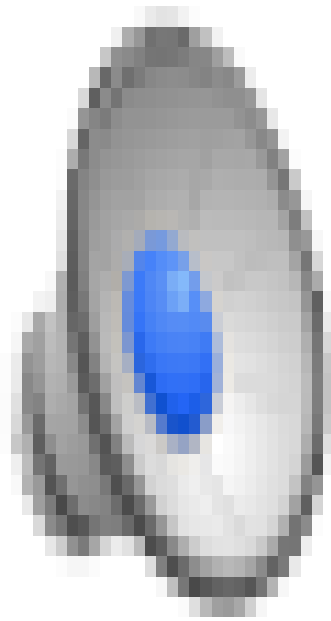
- Strategy: expand a node that you think is closest to a goal state
 - Heuristic: estimate of distance to nearest goal for each state
- A common case:
 - Best-first takes you straight to the (wrong) goal
- Worst-case: like a badly-guided DFS



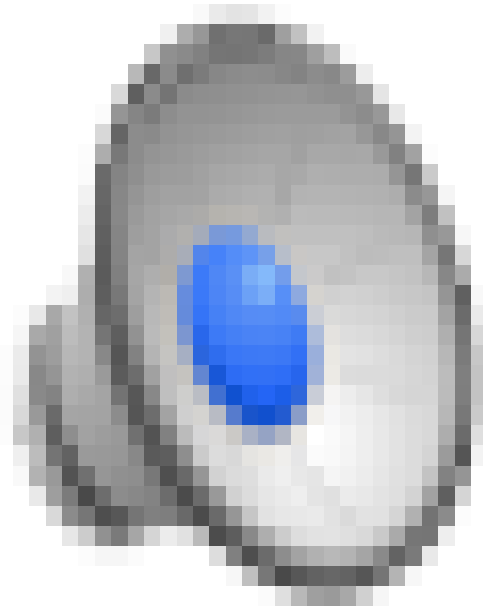
[Demo: contours greedy empty (L3D1)]

[Demo: contours greedy pacman small maze (L3D4)]

Video of Demo Contours Greedy (Empty)



Video of Demo Contours Greedy (Pacman Small Maze)



A* Search



A* Search



UCS

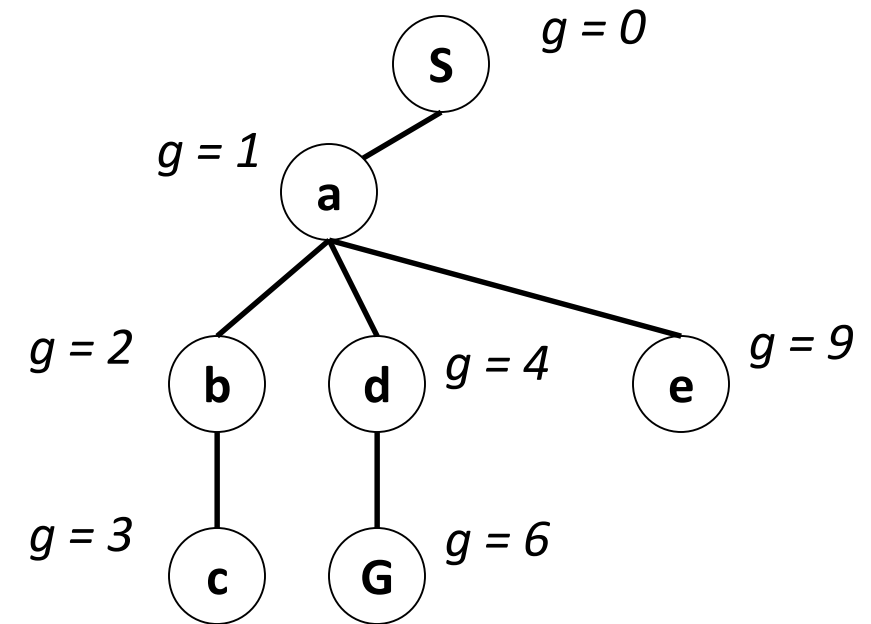
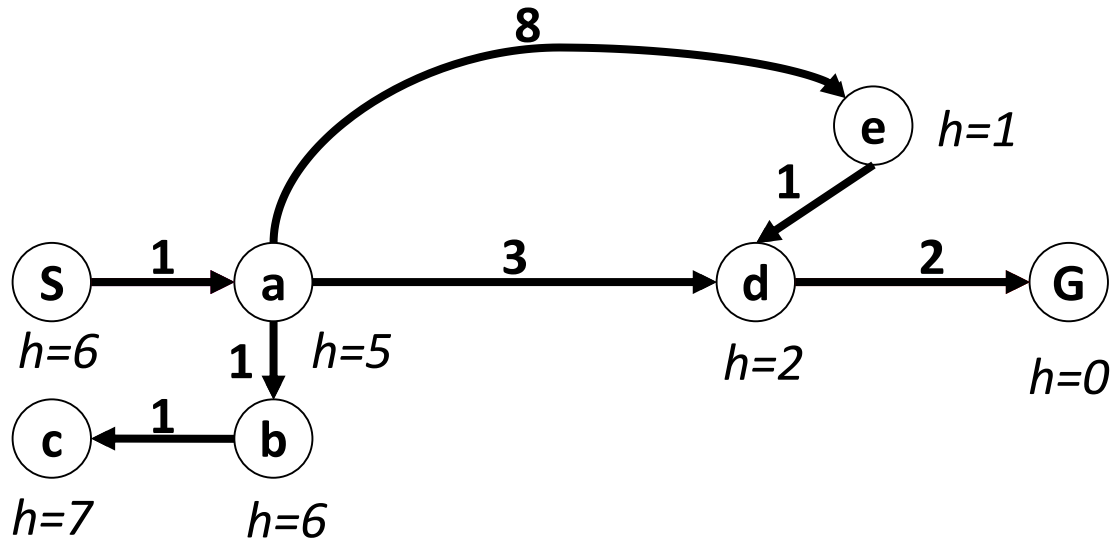


Greedy

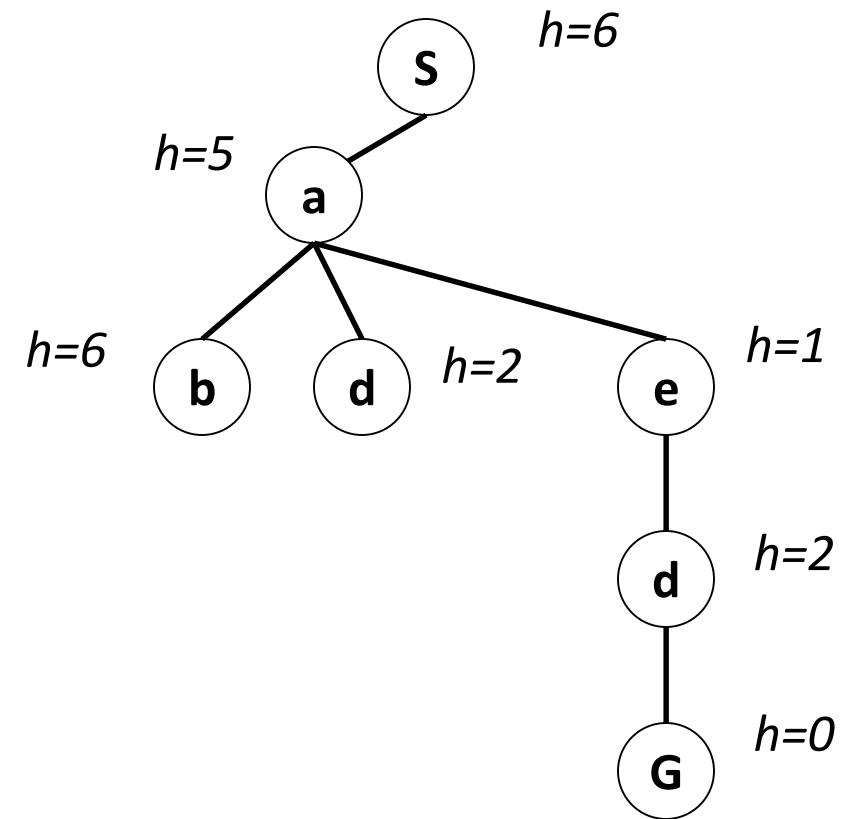
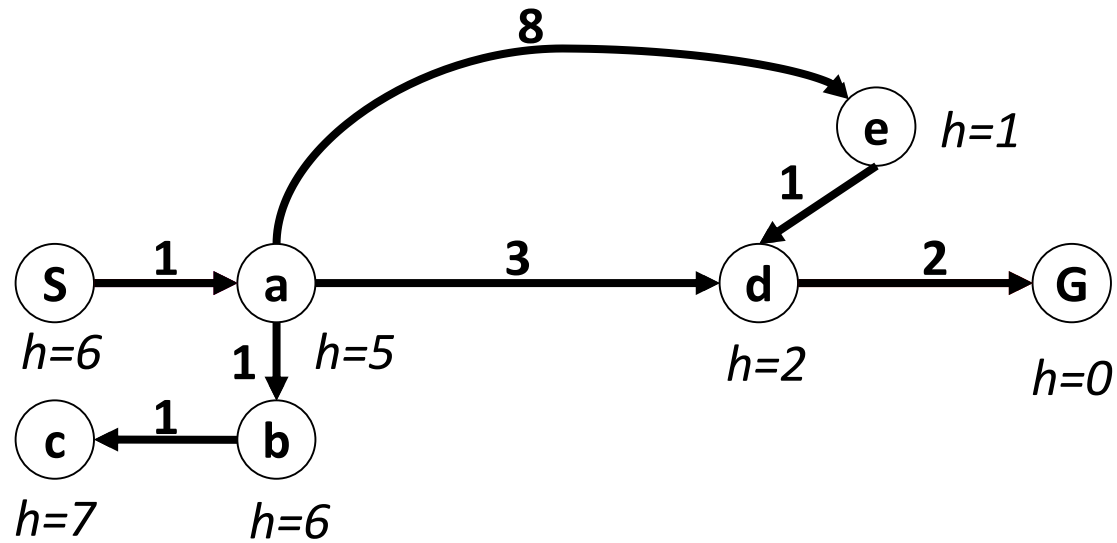


A*

Uniform-Cost Search

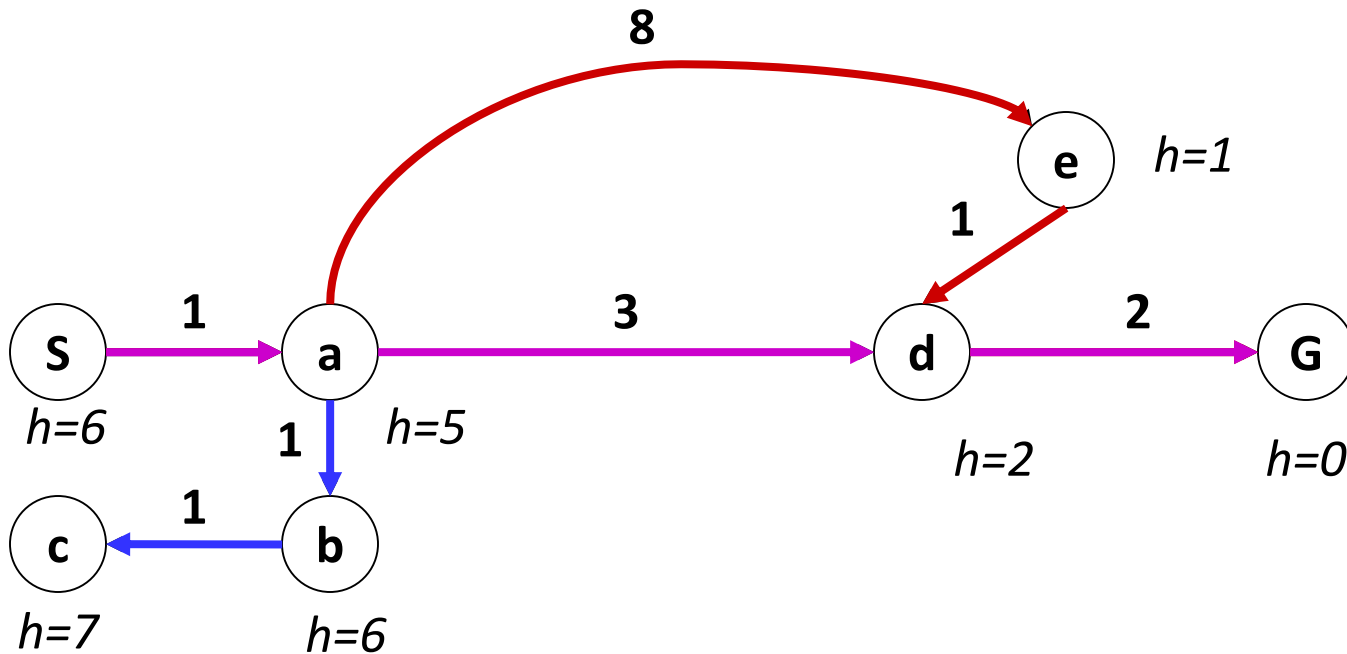


Greedy Search



Combining UCS and Greedy

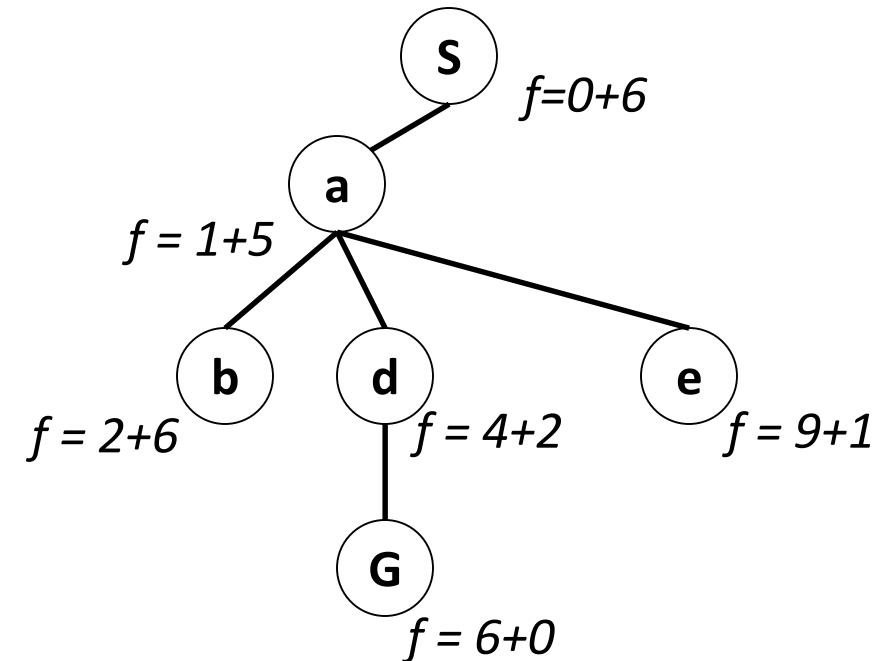
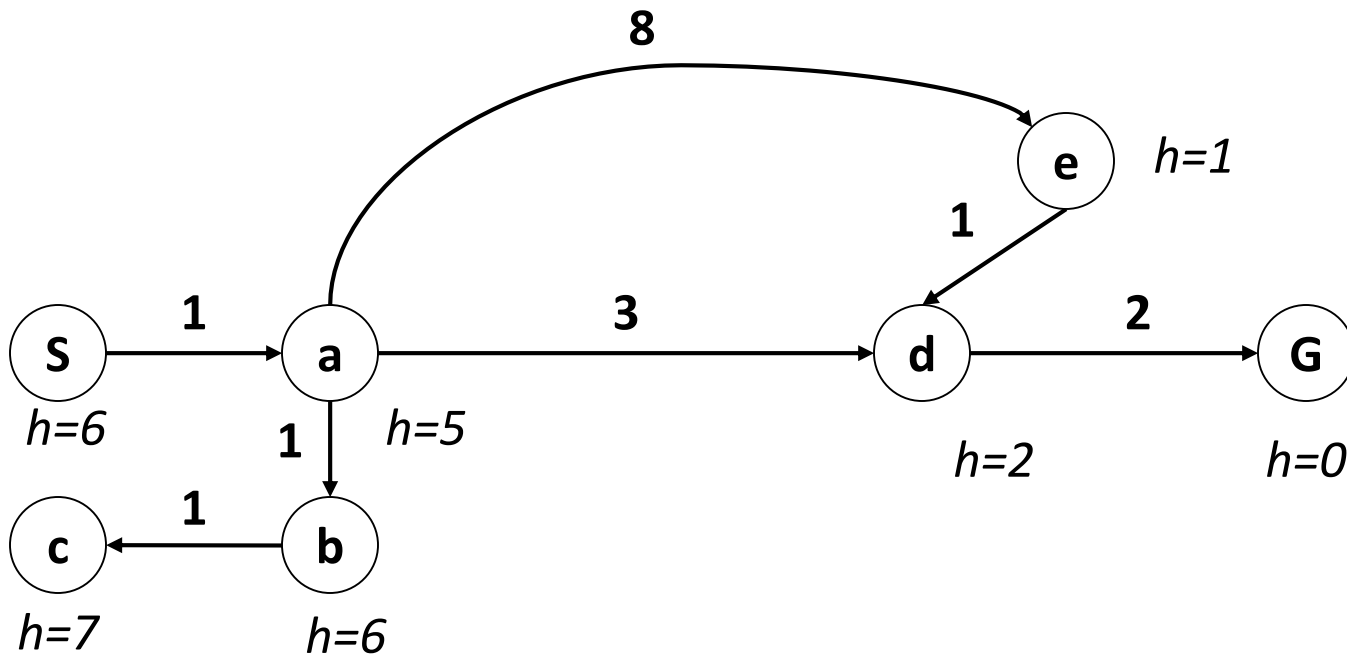
- **Uniform-cost** orders by path cost, or *backward cost* $g(n)$
- **Greedy** orders by goal proximity, or *forward cost* $h(n)$



- **A* Search** orders by the sum: $f(n) = g(n) + h(n)$

Combining UCS and Greedy

- **Uniform-cost** orders by path cost, or *backward cost* $g(n)$
- **Greedy** orders by goal proximity, or *forward cost* $h(n)$



- **A* Search** orders by the sum: $f(n) = g(n) + h(n)$

A* Search

Minimizing the total estimated solution cost

Idea: Avoid expanding paths that are already expensive

Evaluation function $f(n) = g(n) + h(n)$

$g(n)$ = cost so far to reach node n

$h(n)$ = estimated cost to goal from node n

$f(n)$ = estimated total cost of path through node n to goal

- Since $g(n)$ gives the path cost from the start node to node n , and *$h(n)$ is the estimated cost of the cheapest path from n to the goal*,

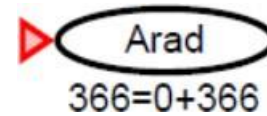
$f(n)$ = estimated cost of the cheapest solution through n

- *If heuristic function $h(n)$ satisfies certain conditions, A* search is both complete and optimal.*

A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$

Arad is the initial state.



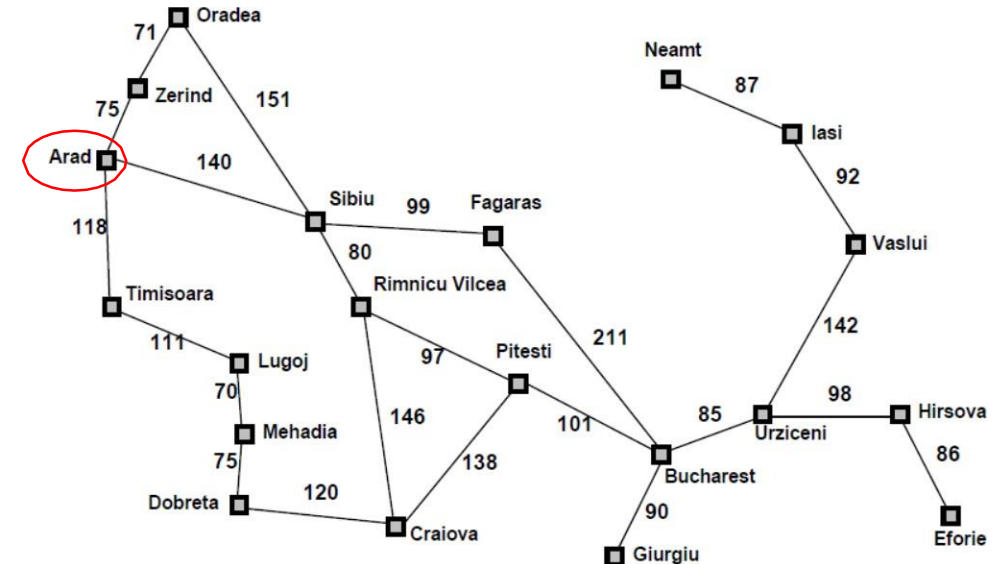
*straight-line distances
to Bucharest*

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

Arad 366

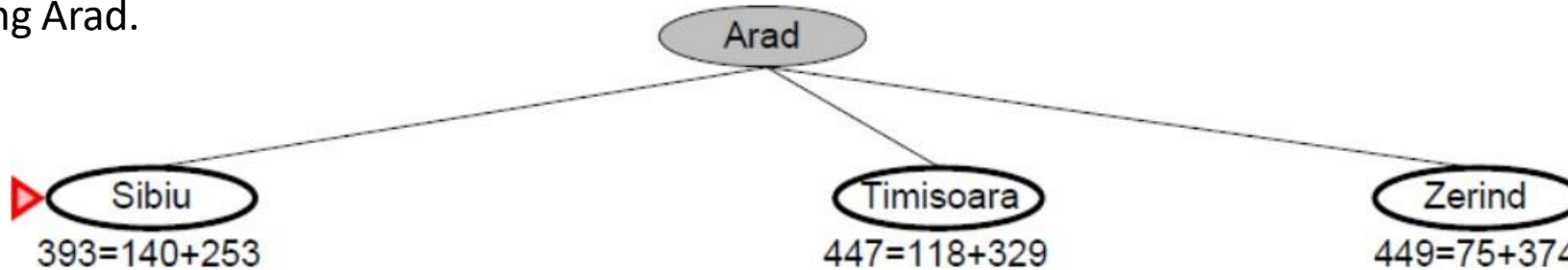
Explored



A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$

After expanding Arad.



straight-line distances
to Bucharest

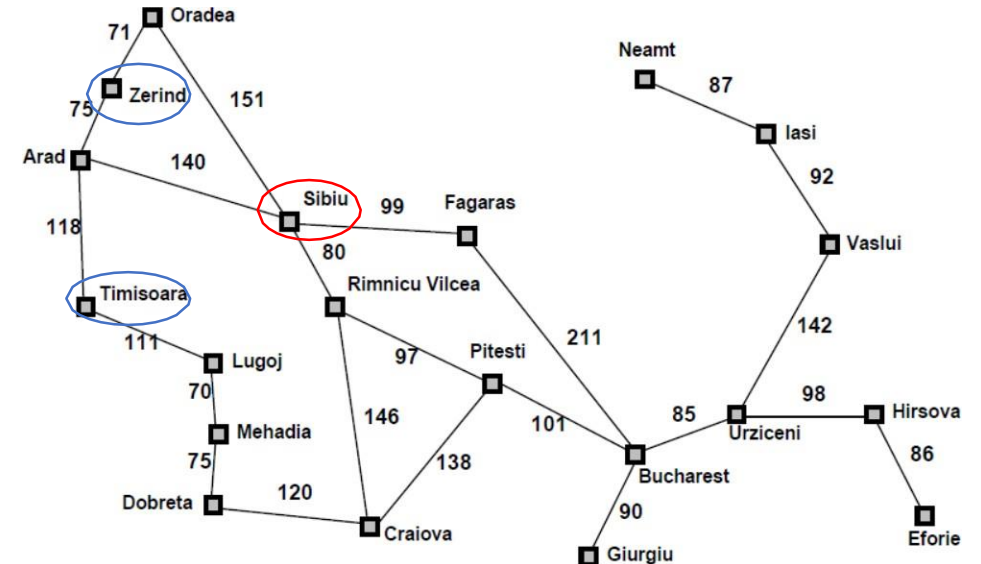
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

Sibiu	393
Timisoara	447
Zerind	449

Explored

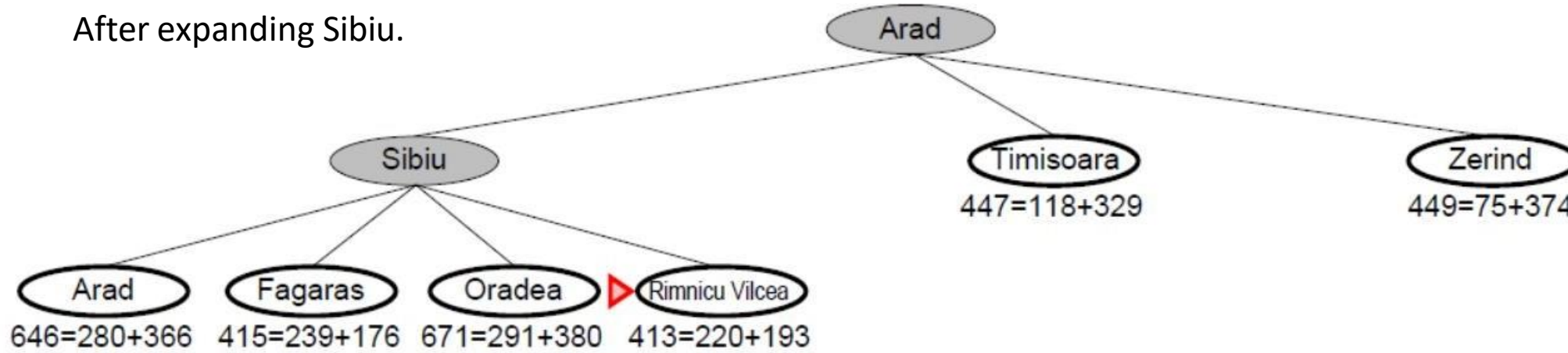
Arad



A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$

After expanding Sibiu.



straight-line distances
to Bucharest

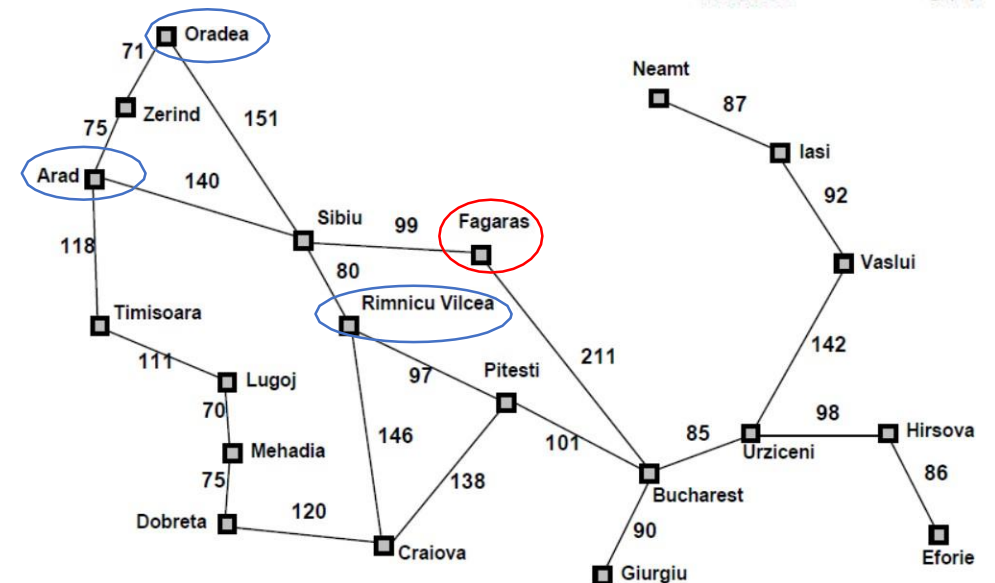
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

RimnicuVil	413
Fagaras	415
Timisoara	447
Zerind	449
Oradea	671

Explored

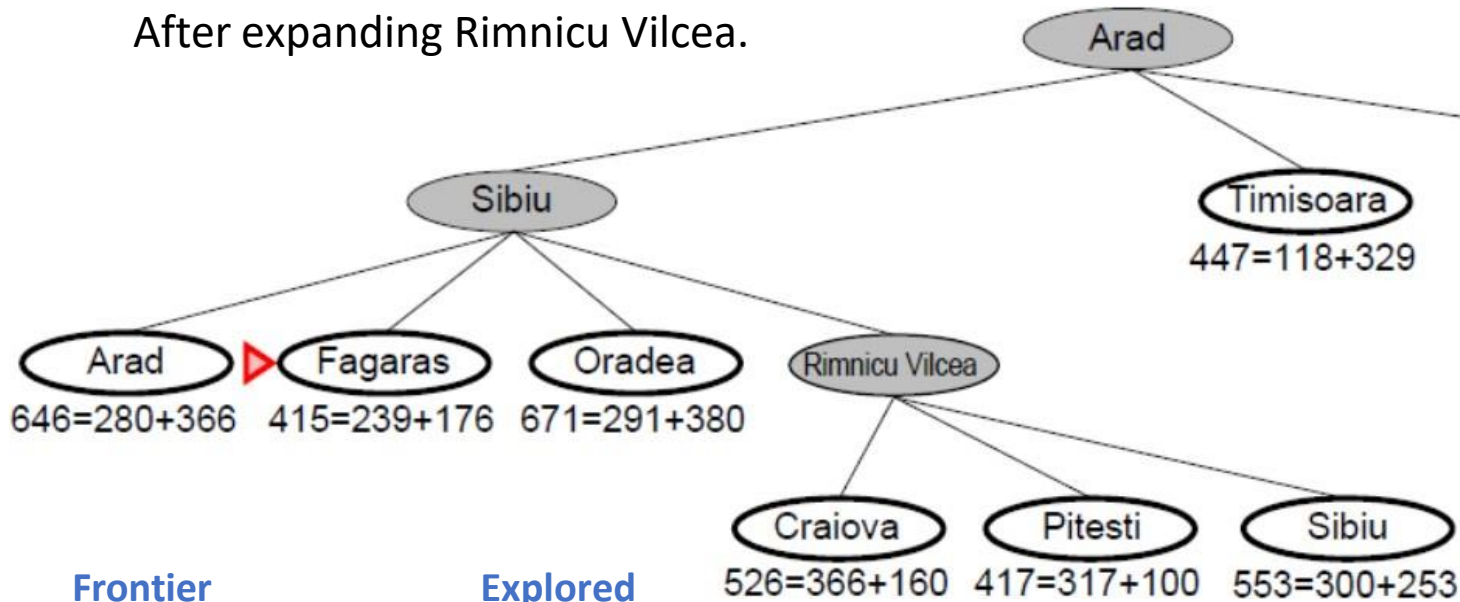
Arad
Sibiu



A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$

After expanding Rimnicu Vilcea.



Frontier

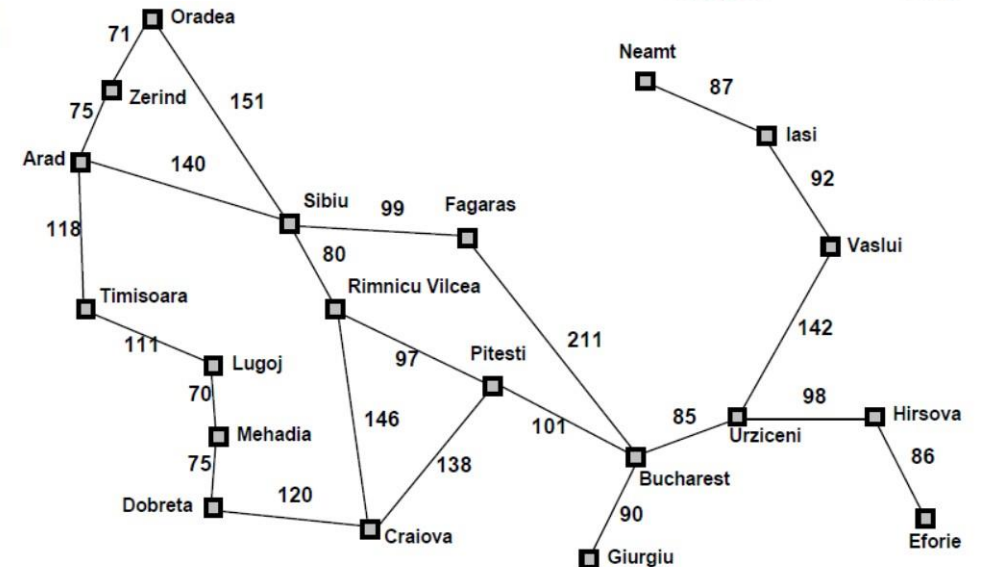
Fagaras 415
Pitesti 417
Timisoara 447
Zerind 449
Craiova 526
Oradea 671

Explored

Arad
Sibiu
RimnicuVil

straight-line distances
to Bucharest

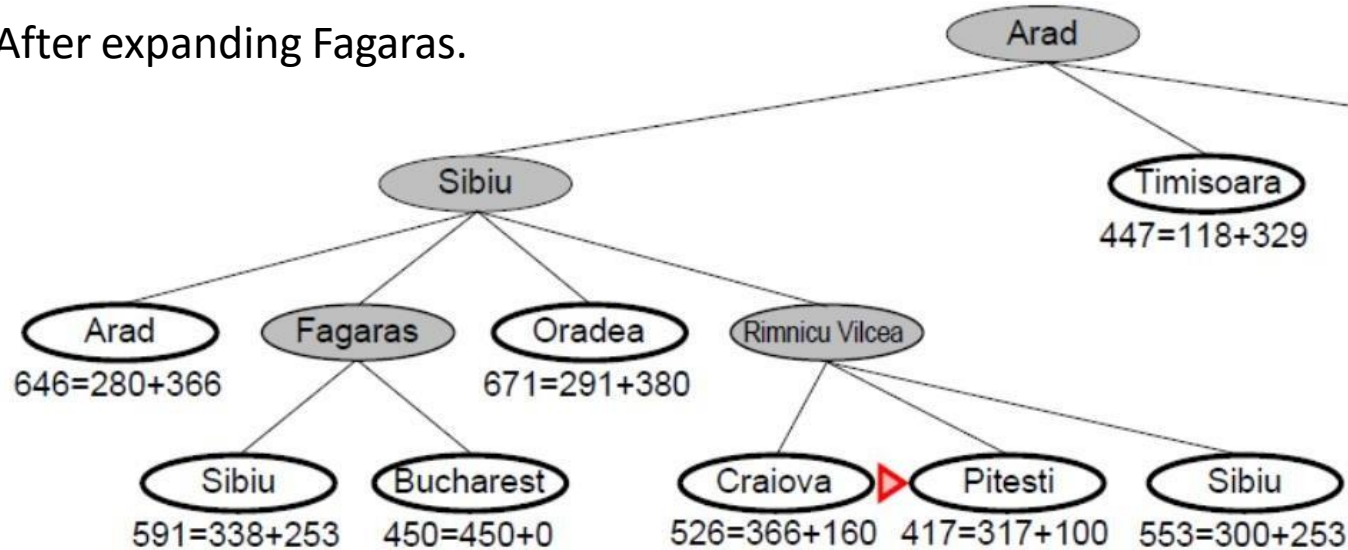
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$

After expanding Fagaras.



straight-line distances
to Bucharest

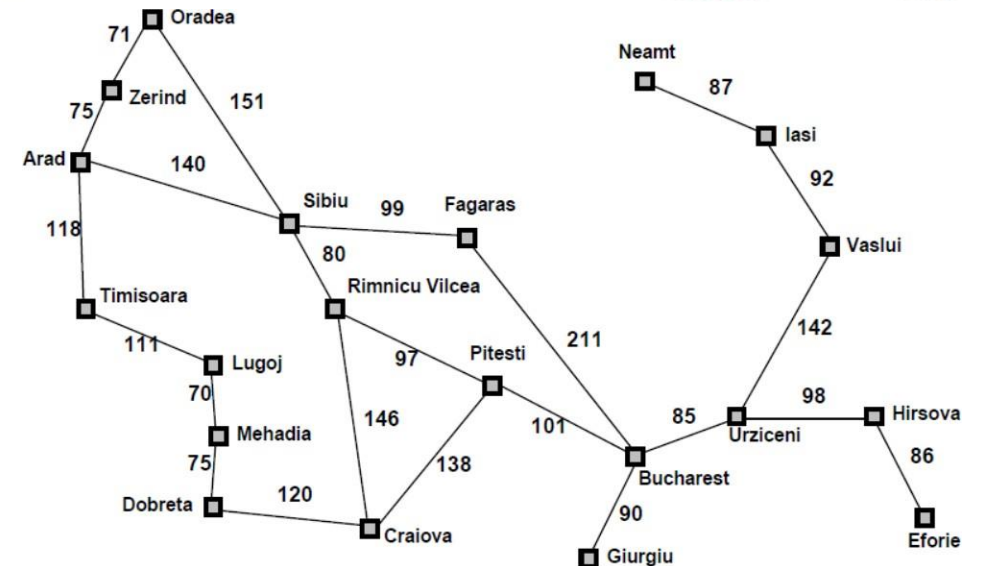
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

Pitesti	417
Timisoara	447
Zerind	449
Bucharest	450
Craiova	526
Oradea	671

Explored

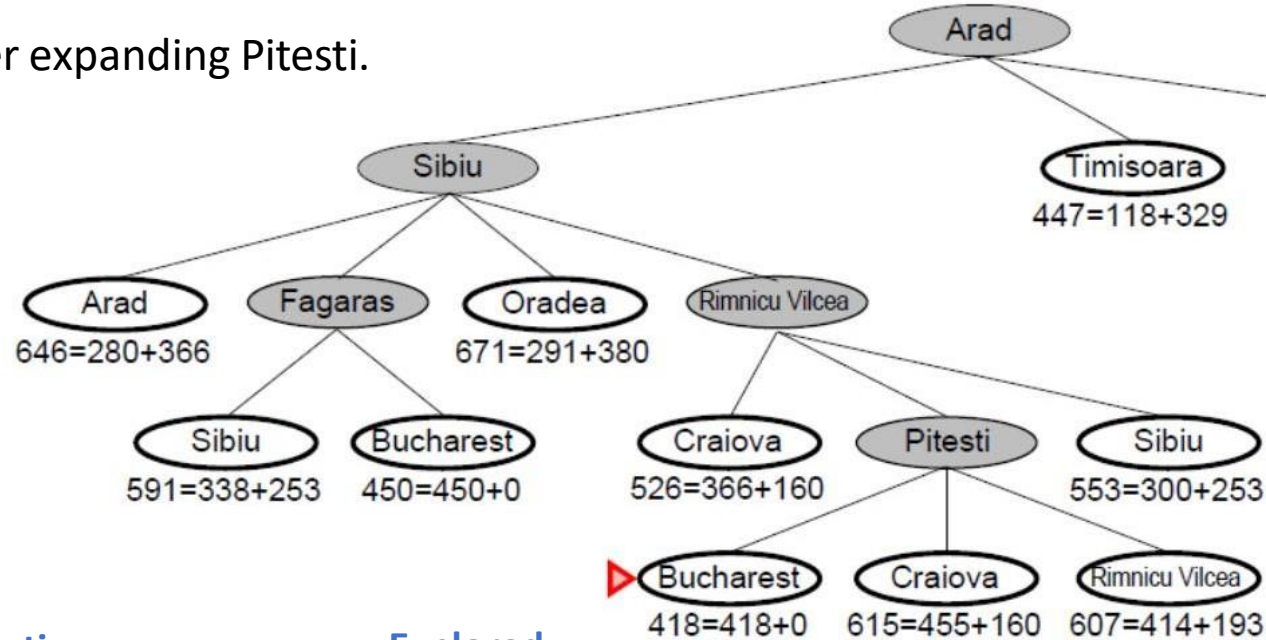
Arad
Sibiu
RimnicuVil
Fagaras



A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$

After expanding Pitesti.



straight-line distances to Bucharest

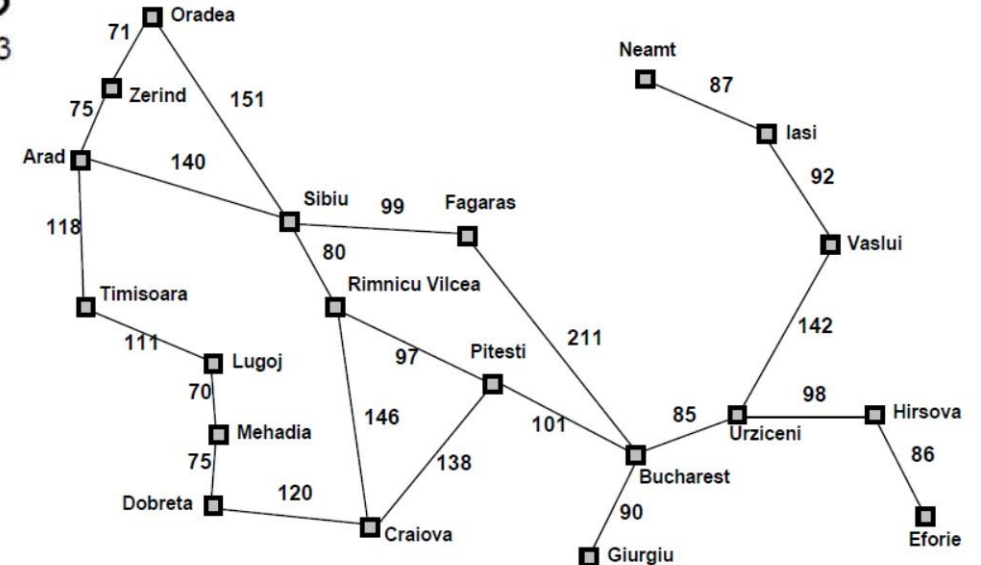
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Frontier

Bucharest	450 418
Timisoara	447
Zerind	449
Craiova	526
Oradea	671

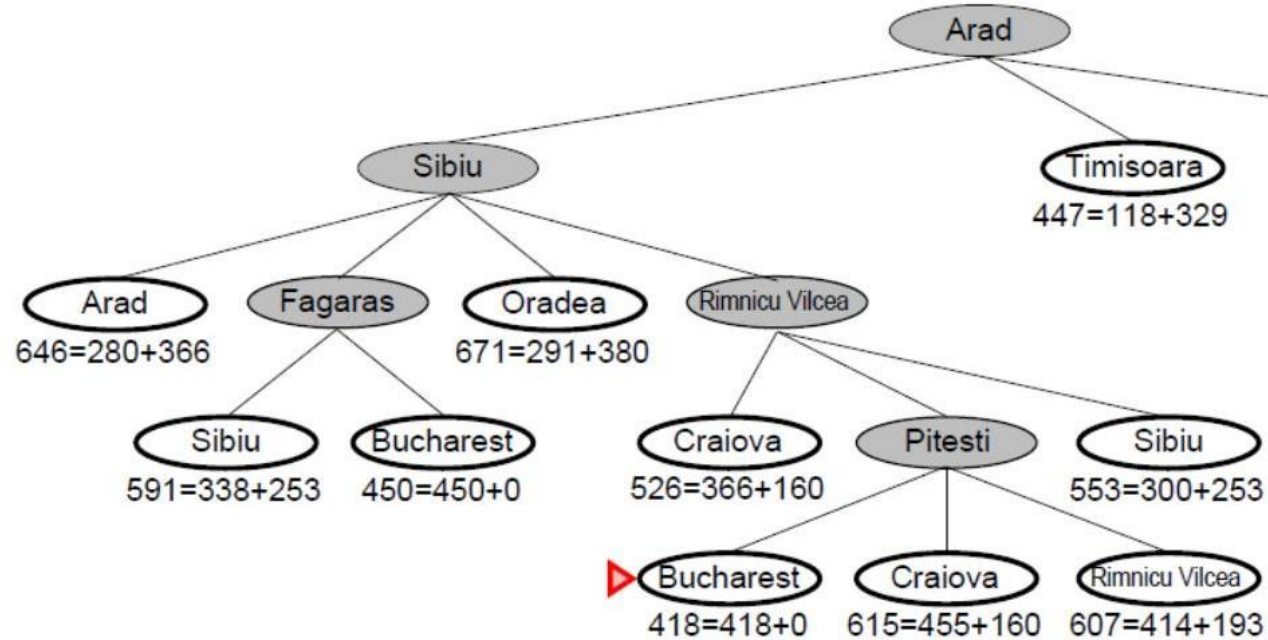
Explored

Arad
Sibiu
RimnicuVil
Fagaras
Pitesti



A* Search Example

Node labels are $f(n) = g(n) + h_{SLD}(n)$



straight-line distances
to Bucharest

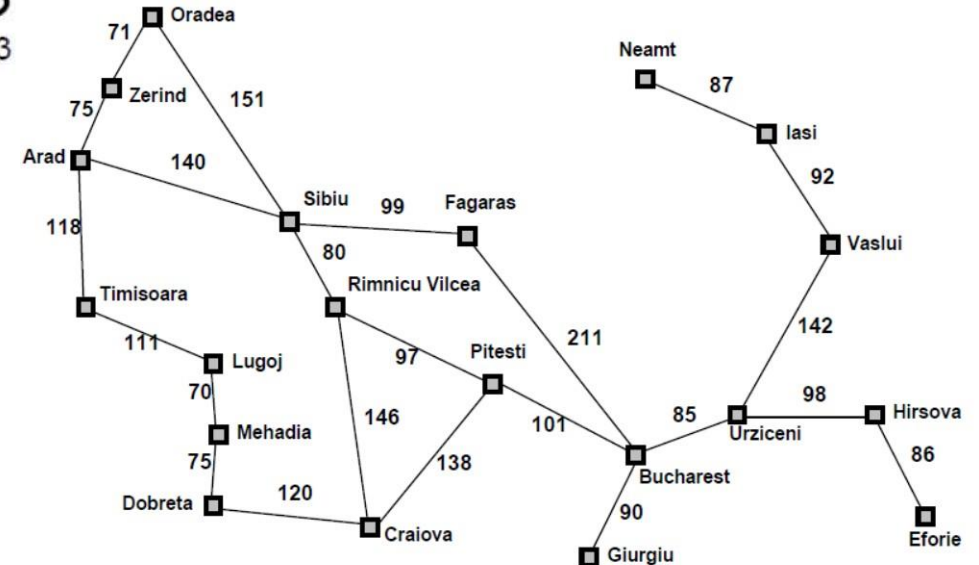
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Path found by A*: Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest

A* Path Cost: $140+80+97+101 = 418$

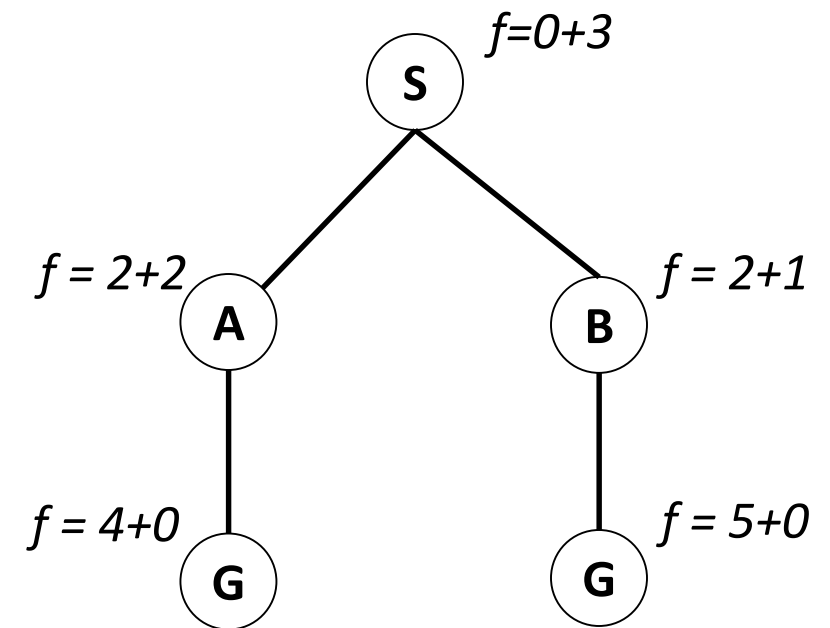
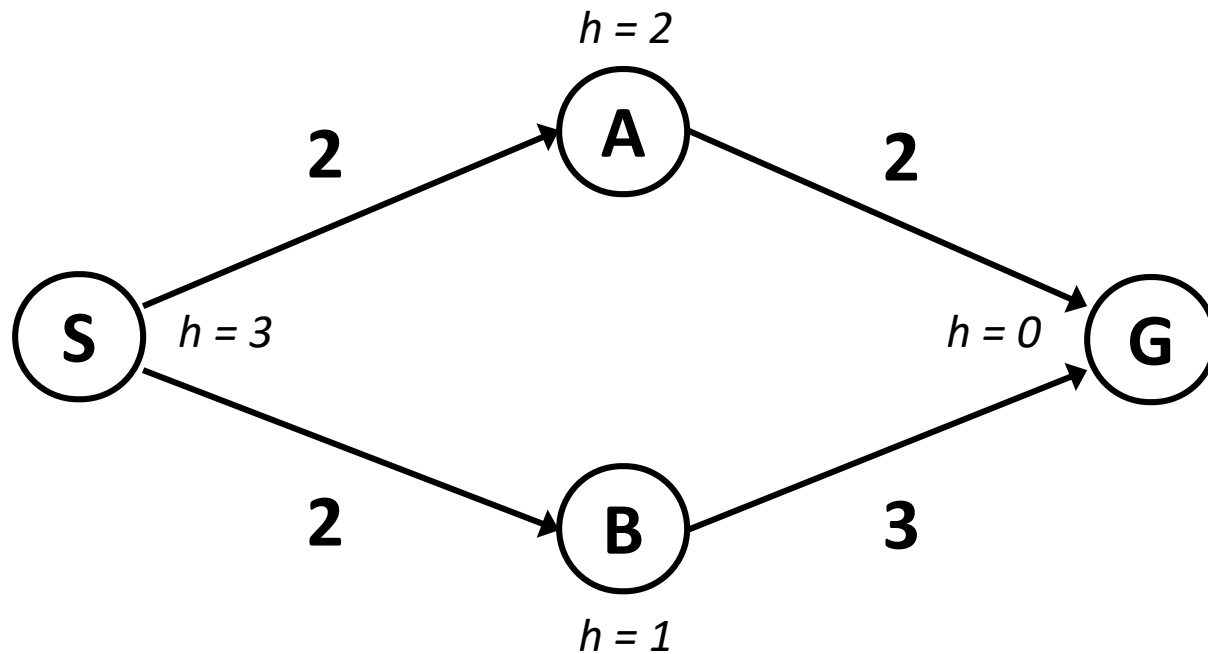
Optimum Path Cost: 418

A* finds an optimum path.



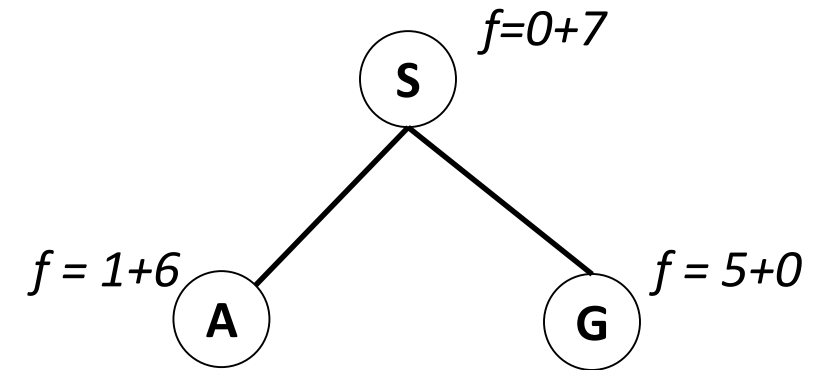
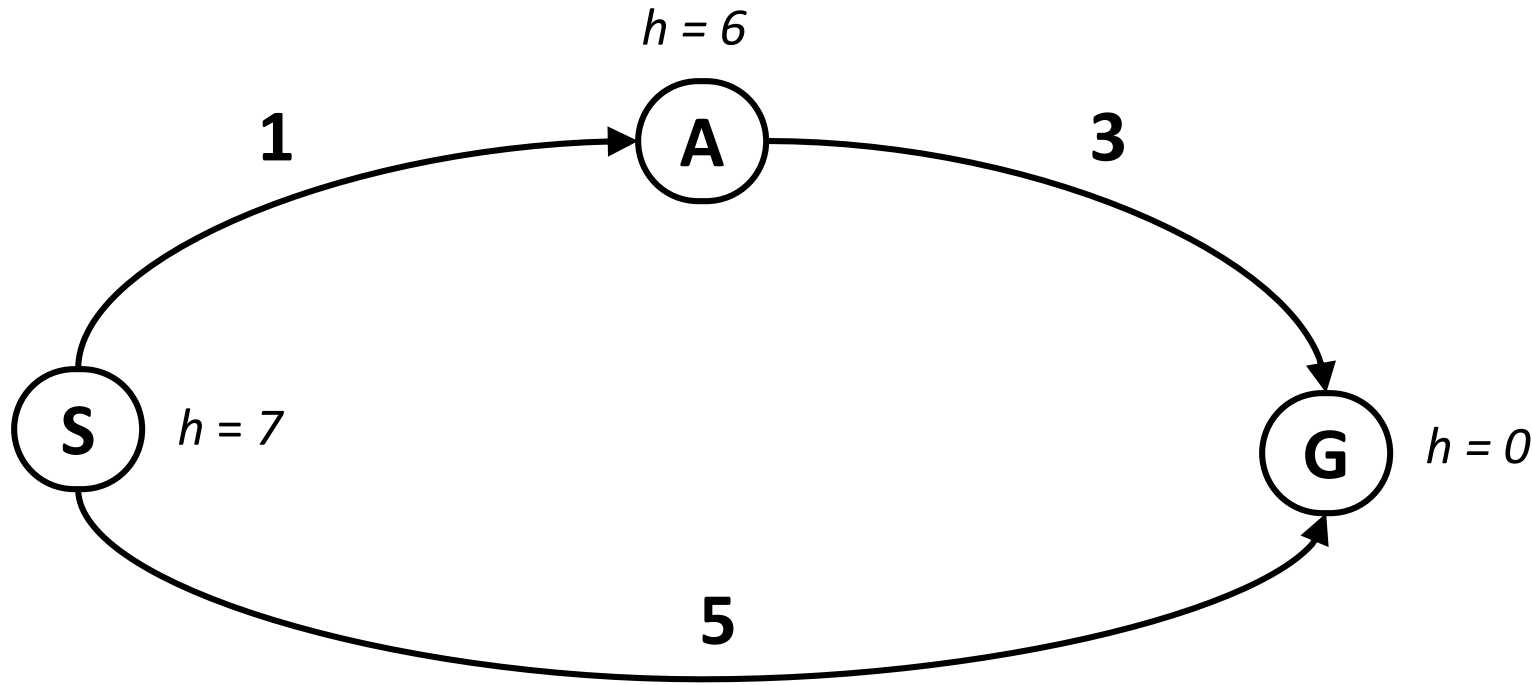
When should A* terminate?

- Should we stop when we enqueue a goal?



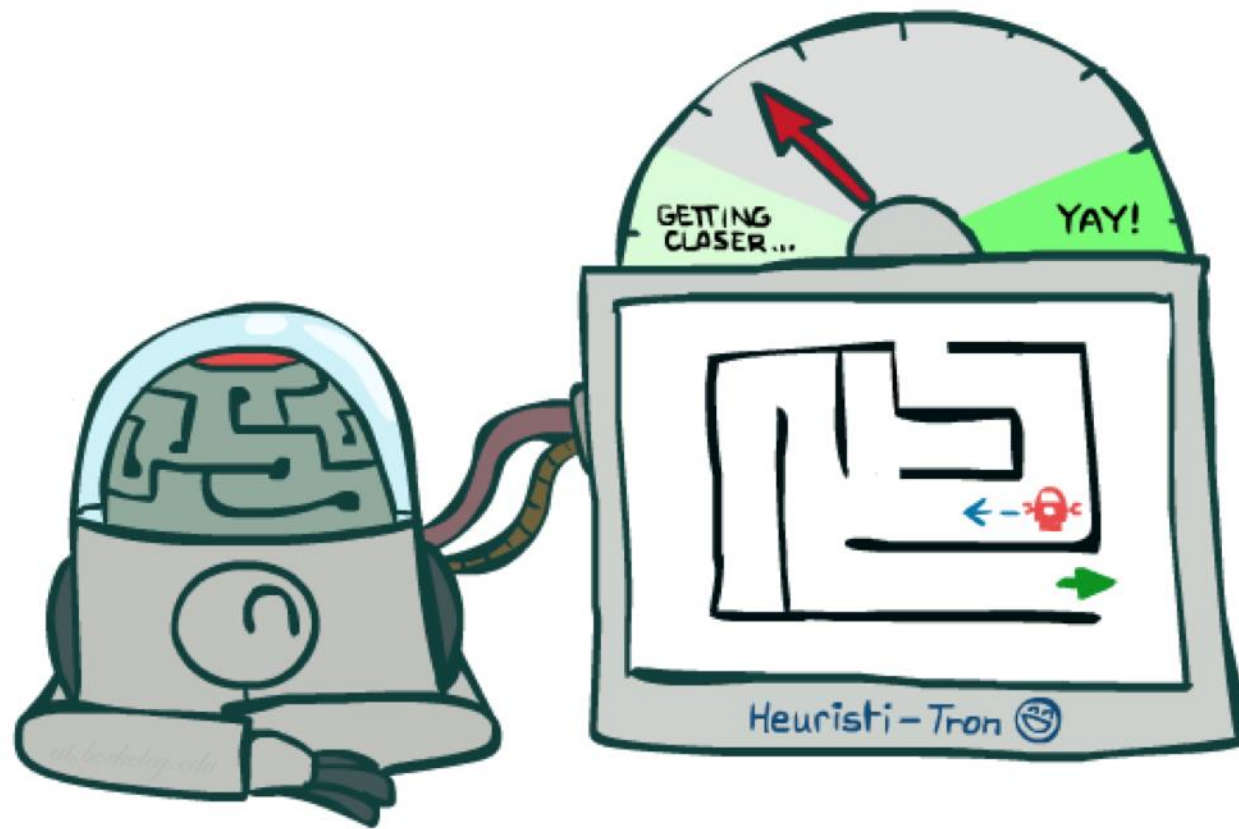
- No: only stop when we dequeue a goal

Is A* Optimal?

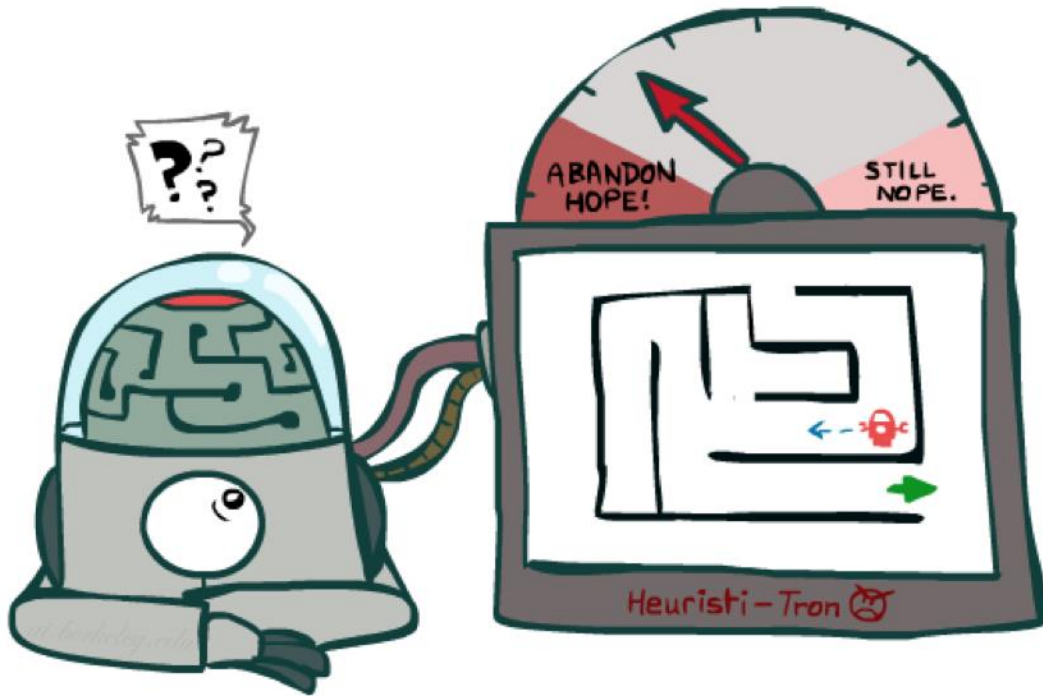


- What went wrong?
- Actual bad goal cost < estimated good goal cost
- We need estimates to be less than actual costs!

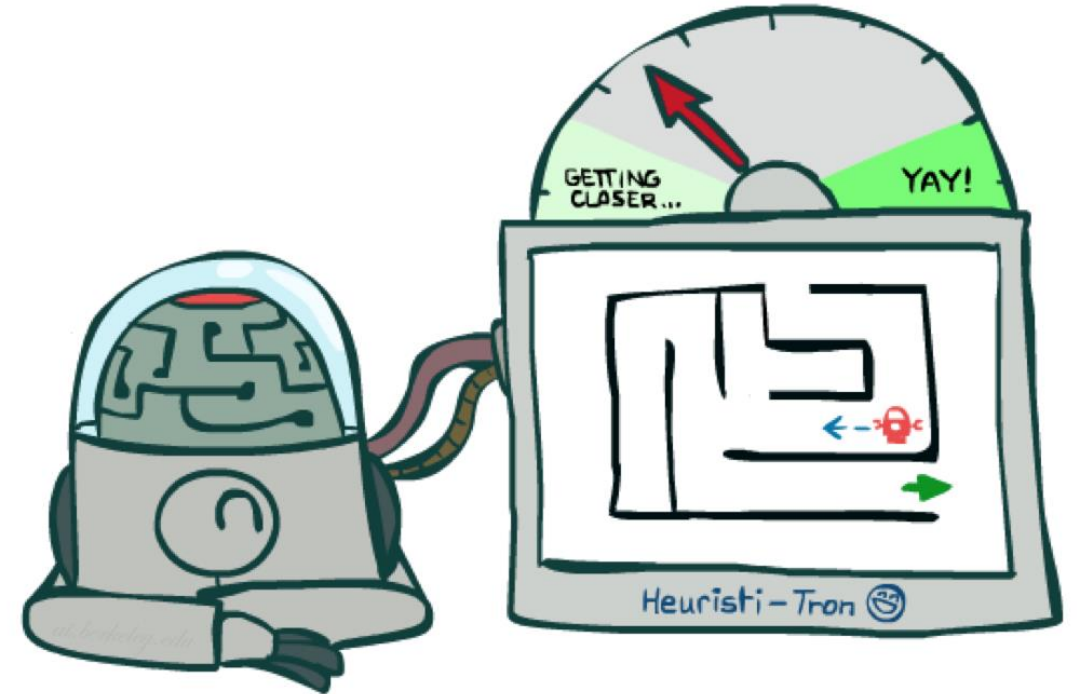
Admissible Heuristics



Idea: Admissibility



Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe



Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

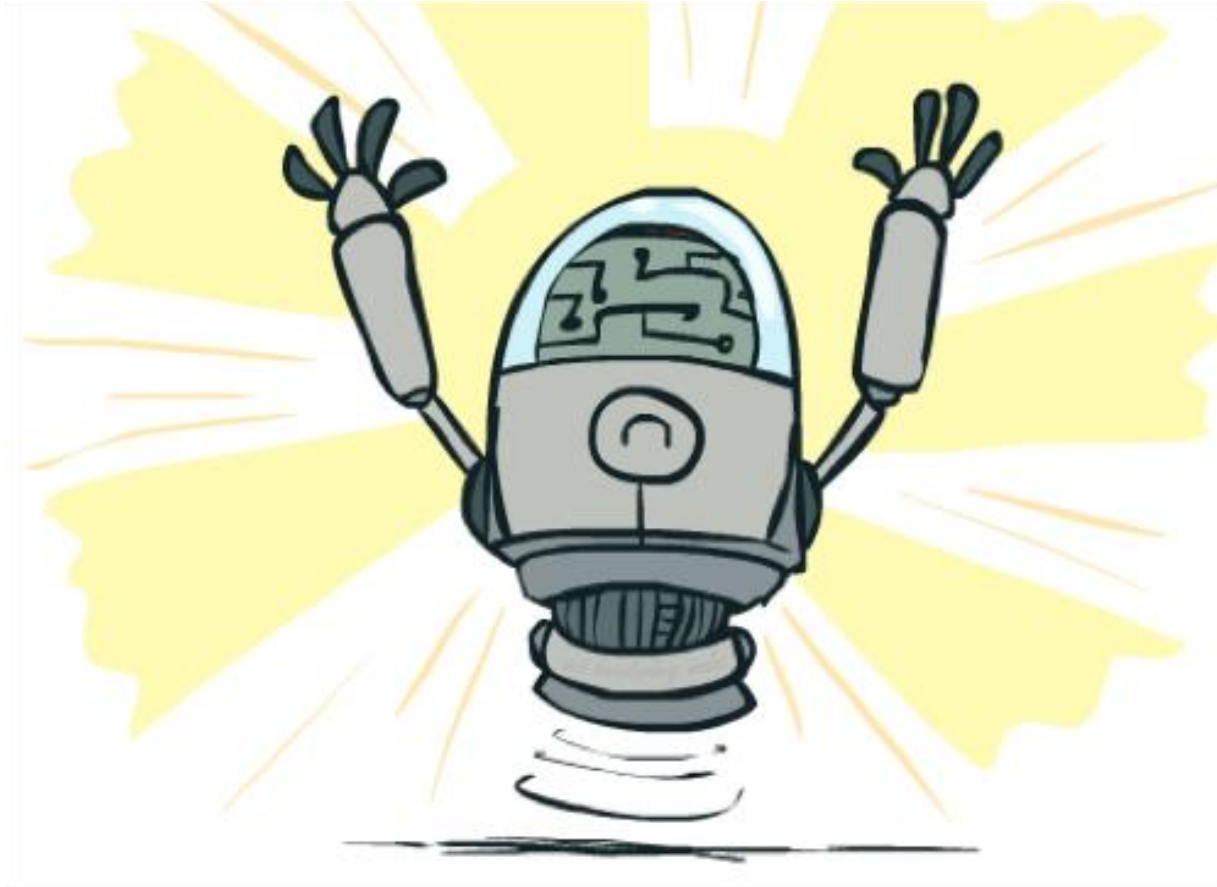
Conditions for Optimality: Admissibility and Consistency

- The first *condition for optimality* is that heuristic function $h(n)$ must be an **admissible heuristic**.
- An **admissible heuristic** is one that never overestimates the cost to reach the goal (optimistic).
- A **heuristic $h(n)$ is admissible** if for every node n , $h(n) \leq C(n)$ where $C(n)$ is the true cost to reach the goal state from the state of node n .
 - **Straight-line distance heuristic $h_{SLD}(n)$ is admissible** because the shortest path between any two points is a straight line. $h_{SLD}(n)$ never overestimates actual distance.
 - If $h(n)$ is admissible, $f(n)$ never overestimates the cost to reach the goal because $f(n) = g(n) + h(n)$ and $g(n)$ is the actual cost to reach node n .
- A **heuristic $h(n)$ is consistent** if, for every node n and every successor n' of n generated by any action a , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n' :

$$h(n) \leq c(n, a, n') + h(n')$$

- $h_{SLD}(n)$ is consistent.
- If $h(n)$ is admissible and consistent, then A^* is complete and optimal.

Optimality of A* Tree Search



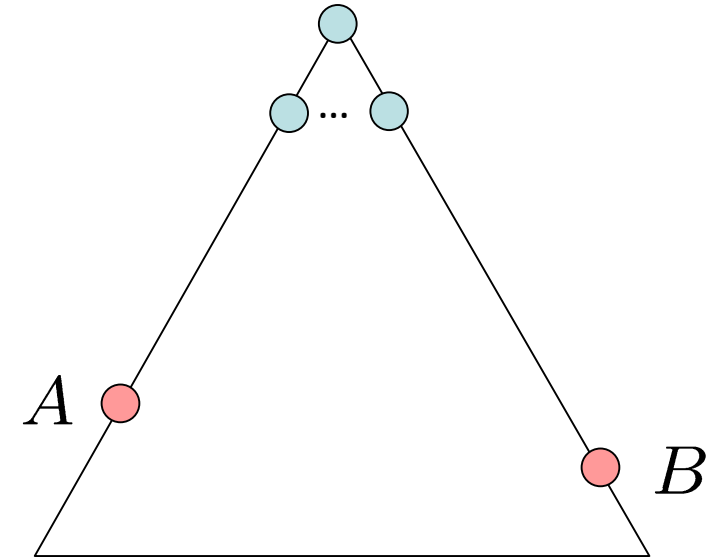
Optimality of A* Tree Search

Assume:

- A is an optimal goal node
- B is a suboptimal goal node
- h is admissible

Claim:

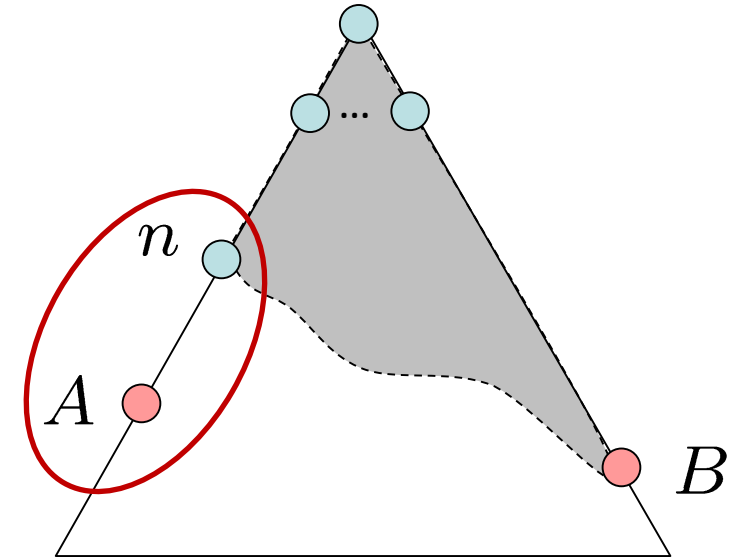
- A will exit the fringe before B



Optimality of A* Tree Search: Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe, too (maybe A !)
- Claim: n will be expanded before B
 1. $f(n)$ is less or equal to $f(A)$



Optimality of A* Tree Search: Blocking

1. $f(n)$ is less than or equal to $f(A)$

- Definition of f-cost says:

$$f(n) = g(n) + h(n) = (\text{path cost to } n) + (\text{est. cost of } n \text{ to } A)$$

$$f(A) = g(A) + h(A) = (\text{path cost to } A) + (\text{est. cost of } A \text{ to } A)$$

- The admissible heuristic must underestimate the true cost

$$h(A) = (\text{est. cost of } A \text{ to } A) = 0$$

- So now, we have to compare:

$$f(n) = g(n) + h(n) = (\text{path cost to } n) + (\text{est. cost of } n \text{ to } A)$$

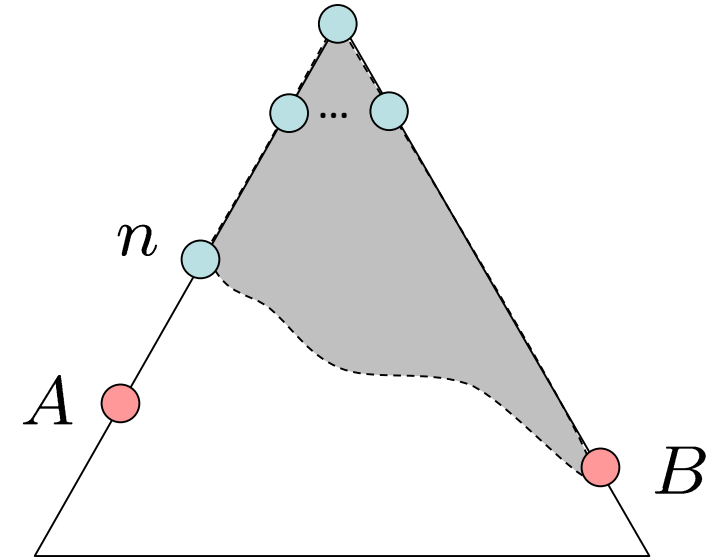
$$f(A) = g(A) = (\text{path cost to } A)$$

- $h(n)$ must be an underestimate of the true cost from n to A

$$(\text{path cost to } n) + (\text{est. cost of } n \text{ to } A) \leq (\text{path cost to } A)$$

$$g(n) + h(n) \leq g(A)$$

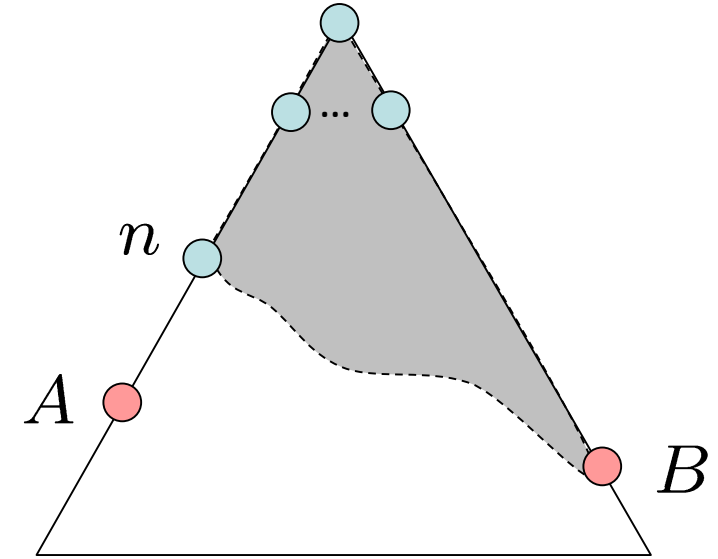
$$f(n) \leq f(A)$$



Optimality of A* Tree Search: Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe, too (maybe A !)
- Claim: n will be expanded before B
 1. $f(n)$ is less or equal to $f(A)$
 2. $f(A)$ is less than $f(B)$



Optimality of A* Tree Search: Blocking

2. $f(A)$ is less than $f(B)$

- We know that:

$$f(A) = g(A) + h(A) = (\text{path cost to } A) + (\text{est. cost of } A \text{ to } A)$$

$$f(B) = g(B) + h(B) = (\text{path cost to } B) + (\text{est. cost of } B \text{ to } B)$$

- The heuristic must underestimate the true cost:

$$h(A) = h(B) = 0$$

- So now, we have to compare:

$$f(A) = g(A) = (\text{path cost to } A)$$

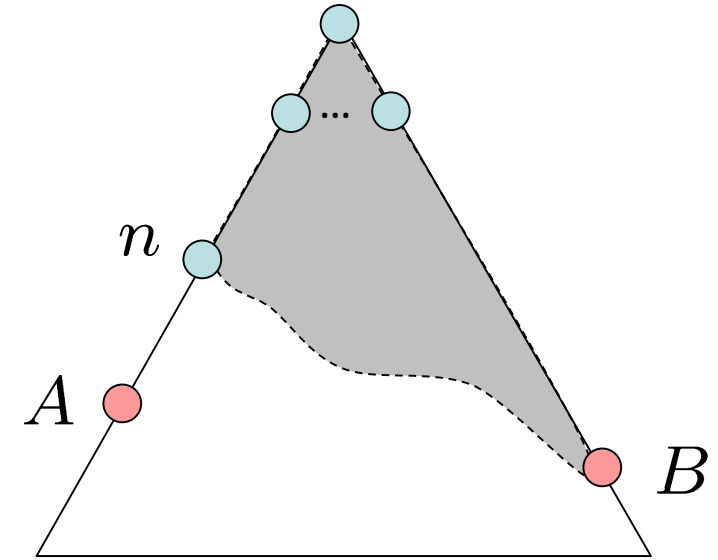
$$f(B) = g(B) = (\text{path cost to } B)$$

- We assumed that B is suboptimal! So

$$(\text{path cost to } A) < (\text{path cost to } B)$$

$$g(A) < g(B)$$

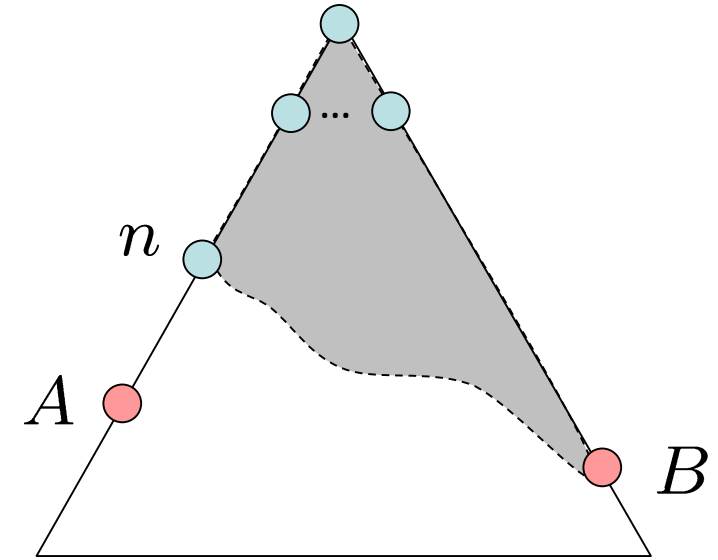
$$f(A) < f(B)$$



Optimality of A* Tree Search: Blocking

Proof:

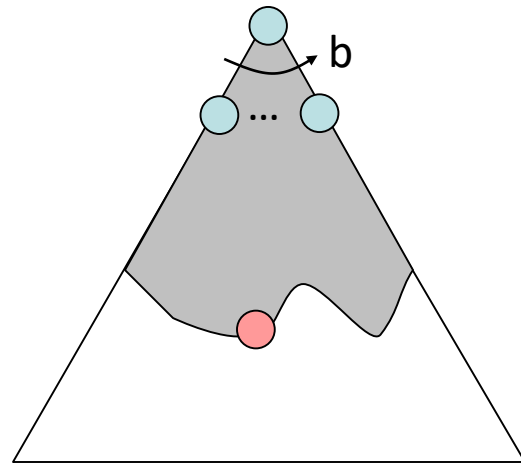
- Imagine B is on the fringe
- Some ancestor n of A is on the fringe, too (maybe A!)
- Claim: n will be expanded before B
 1. $f(n)$ is less or equal to $f(A)$
 2. $f(A)$ is less than $f(B)$
 3. n expands before B
- All ancestors of A expand before B
- A expands before B
- A* search is optimal



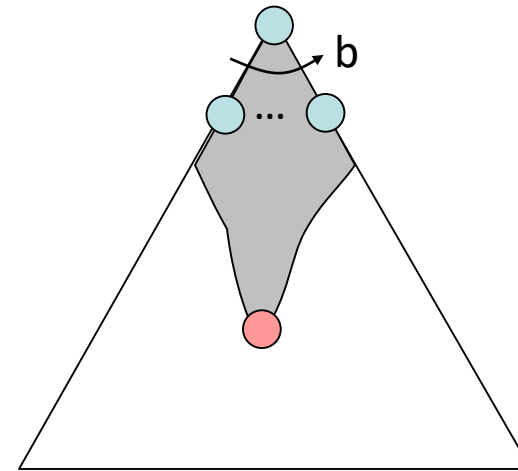
Properties of A^*

Properties of A^*

Uniform-Cost

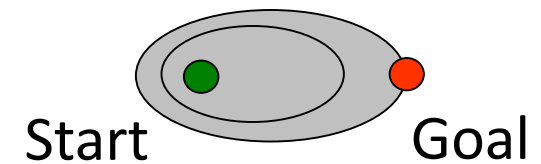
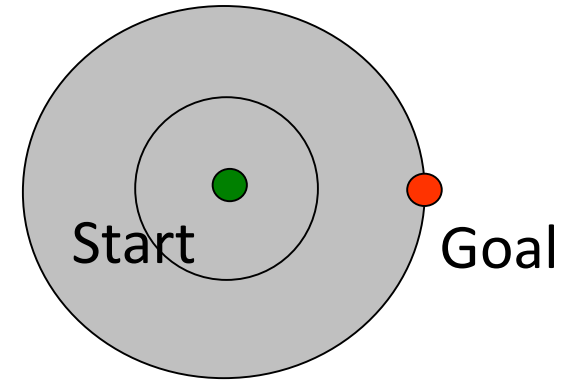


A^*



UCS vs A* Contours

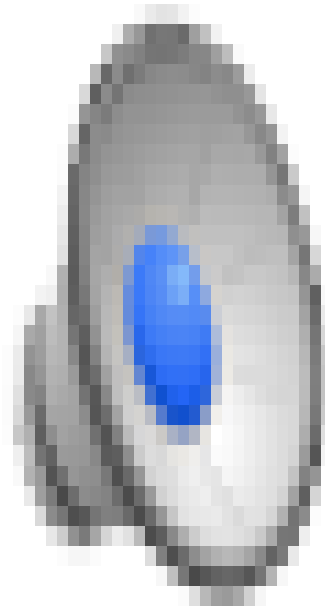
- Uniform-cost expands equally in all “directions”
- A* expands mainly toward the goal, but does hedge its bets to ensure optimality



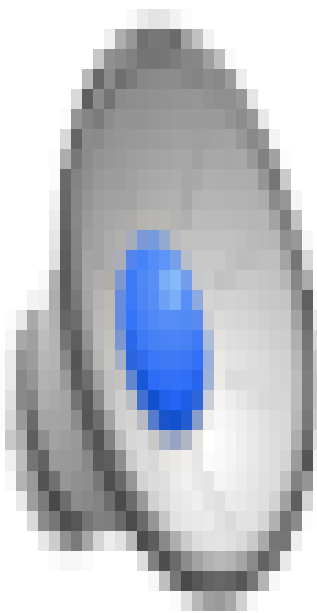
[Demo: contours UCS / greedy / A* empty (L3D1)]

[Demo: contours A* pacman small maze (L3D5)]

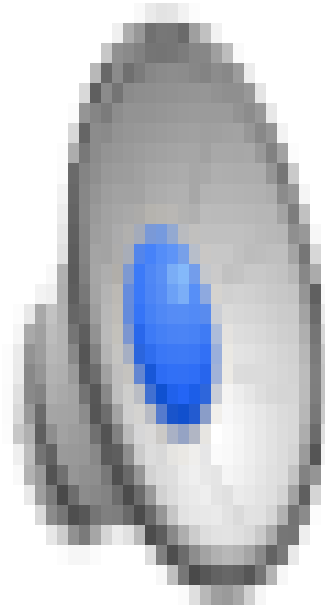
Video of Demo Contours (Empty) -- UCS



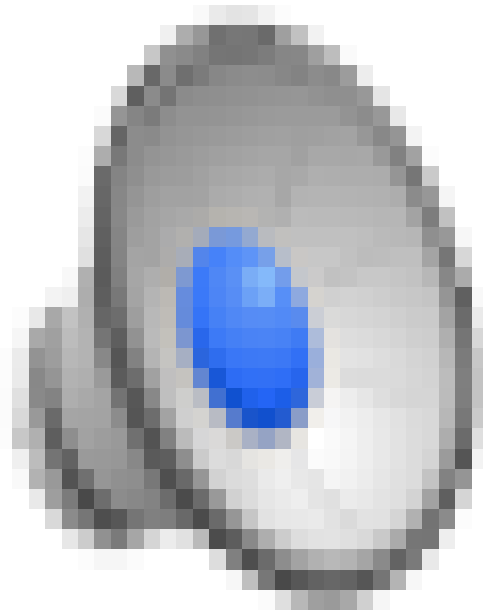
Video of Demo Contours (Empty) -- Greedy



Video of Demo Contours (Empty) – A^*



Video of Demo Contours (Pacman Small Maze) – A*



Comparison



Greedy



Uniform Cost



A*

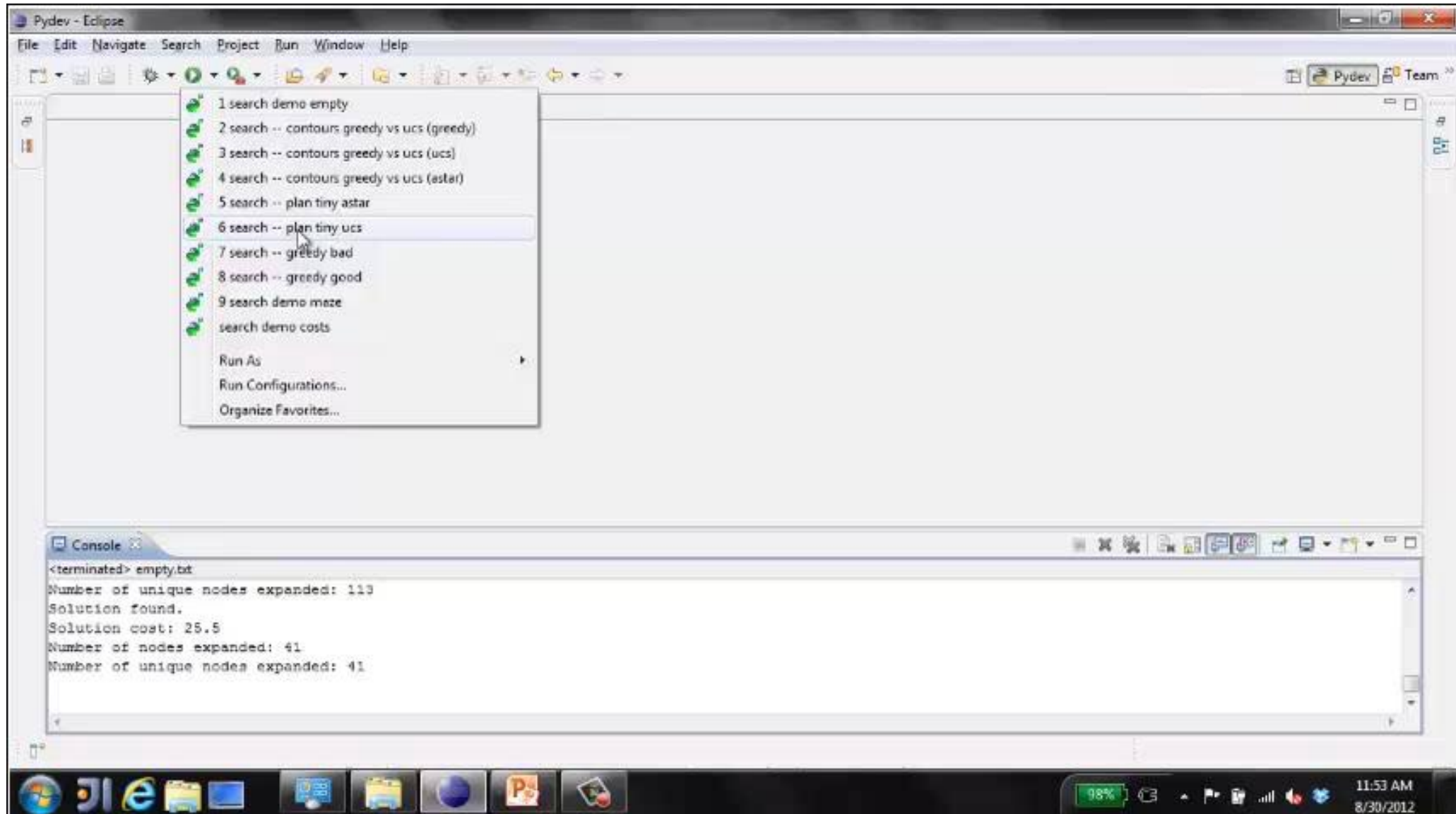
A* Applications

- Video games
- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Machine translation
- Speech recognition
- ...



[Demo: UCS / A* pacman tiny maze (L3D6,L3D7)]
[Demo: guess algorithm Empty Shallow/Deep (L3D8)]

Video of Demo Pacman (Tiny Maze) – UCS / A*



Video of Demo Empty Water Shallow/Deep – Guess Algorithm

